

Package ‘qgraph’

January 19, 2016

Type Package

Title Graph Plotting Methods, Psychometric Data Visualization and Graphical Model Estimation

Version 1.3.2

Author Sacha Epskamp, Giulio Costantini, Angelique O. J. Cramer, Lourens J. Waldorp, Verena D. Schmittmann and Denny Borsboom

Maintainer Sacha Epskamp <mail@sachaepskamp.com>

Depends R (>= 3.0.0)

Suggests RSVGTipsDevice, sendplot

Imports methods, grDevices, psych, ellipse, lavaan, sem, plyr, Hmisc, igraph, jpeg, png, colorspace, Matrix, sna, corpcor, reshape2, ggplot2, glasso, huge, fdrtool, d3Network, ggm, gtools

ByteCompile yes

Description The qgraph package can be used to visualize data as networks as well as provides an interface for visualizing weighted graphical models.

URL <http://sachaepskamp.com/qgraph>

License GPL-2

LazyLoad yes

NeedsCompilation yes

Repository CRAN

Date/Publication 2016-01-19 15:15:38

R topics documented:

as.igraph.qgraph	2
averageLayout	3
big5	4
big5groups	4
centrality	5
centrality and clustering plots	7

centrality_auto	8
clustcoef_auto	9
cor_auto	12
EBICglasso	13
FDRnetwork	15
findGraph	17
getWmat	18
makeBW	19
mat2vec	20
plot.qgraph	21
print.qgraph	21
qgraph	22
qgraph.animate	38
qgraph.cfa	42
qgraph.efa	44
qgraph.gui	45
qgraph.lavaan	46
qgraph.layout.fruchtermanreingold	49
qgraph.loadings	51
qgraph.panel	53
qgraph.pca	54
qgraph.sem	55
qgraph.semModel	58
qgraph.svg	60
qgraphAnnotate	62
qgraphMixed	63
smallworldness	64
summary.qgraph	66
VARglm	66
wi2net	67
Index	68

as.igraph.qgraph	<i>Converts qgraph object to igraph object.</i>
------------------	---

Description

This function converts the output of [qgraph](#) to an 'igraph' object that can be used in the igraph package (Csardi & Nepusz, 2006)

Usage

```
## S3 method for class 'qgraph'
as.igraph(object, attributes = TRUE)
```

Arguments

object	A "qgraph" object
attributes	Logical, should graphical attributes also be transferred?

Author(s)

Sacha Epskamp <mail@sachaepskamp.com>

References

Csardi G, Nepusz T (2006). The igraph software package for complex network research, InterJournal, Complex Systems 1695. <http://igraph.sf.net>

averageLayout	<i>Computes an average layout over several graphs</i>
---------------	---

Description

This function can be used to compute a joint layout over several graphs.

Usage

```
averageLayout(..., layout = "spring", repulsion = 1, layout.par)
```

Arguments

...	Multiple graph objects such as qgraph objects or weights matrices.
layout	Same as in qgraph
repulsion	The repulsion parameter as used in qgraph .
layout.par	Same as in qgraph

Value

A layout matrix

Author(s)

Sacha Epskamp <mail@sachaepskamp.com>

big5

*Big 5 dataset***Description**

This is a dataset of the Big 5 personality traits that will be used in talks and the paper. It is a measurement of the Dutch translation of the NEO-PI-R on 500 first year psychology students (Dolan, Oort, Stoel, Wicherts, 2009).

Usage

```
data(big5)
```

Format

The format is: num [1:500, 1:240] 2 3 4 4 5 2 2 1 4 2 ... - attr(*, "dimnames")=List of 2 ..\$: NULL ..\$: chr [1:240] "N1" "E2" "O3" "A4" ...

References

Hoekstra, H. A., Ormel, J., & De Fruyt, F. (2003). NEO-PI-R/NEO-FFI: Big 5 persoonlijkheidsvragenlijst. Handleiding [Manual of the Dutch version of the NEO-PI-R/NEO-FFI]. Lisse, The Netherlands: Swets and Zeitlinger.

Dolan, C. V., Oort, F. J., Stoel, R. D., & Wicherts, J. M. (2009). Testing measurement invariance in the target rotates multigroup exploratory factor model. *Structural Equation Modeling*, 16, 295-314.

big5groups

*Big 5 groups list***Description**

This is the groups list of the big 5 data. It is a measurement of the Dutch translation of the NEO-PI-R on 500 first year psychology students (Dolan, Oort, Stoel, Wicherts, 2009).

Usage

```
data(big5groups)
```

Format

The format is: List of 5 \$ Neuroticism : num [1:48] 1 6 11 16 21 26 31 36 41 46 ... \$ Extraversion : num [1:48] 2 7 12 17 22 27 32 37 42 47 ... \$ Openness : num [1:48] 3 8 13 18 23 28 33 38 43 48 ... \$ Agreeableness : num [1:48] 4 9 14 19 24 29 34 39 44 49 ... \$ Conscientiousness: num [1:48] 5 10 15 20 25 30 35 40 45 50 ...

References

- Hoekstra, H. A., Ormel, J., & De Fruyt, F. (2003). NEO-PI-R/NEO-FFI: Big 5 persoonlijkheidsvragenlijst. Handleiding [Manual of the Dutch version of the NEO-PI-R/NEO-FFI]. Lisse, The Netherlands: Swets and Zeitlinger.
- Dolan, C. V., Oort, F. J., Stoel, R. D., & Wicherts, J. M. (2009). Testing measurement invariance in the target rotates multigroup exploratory factor model. *Structural Equation Modeling*, 16, 295-314.

centrality

Centrality statistics of graphs

Description

This function can be used on the output of [qgraph](#) to compute the node centrality statistics for weighted graphs proposed by Opsahl, Agneessens and Skvoretz (2010).

PLEASE USE [centrality_auto](#) FOR A MORE USER FRIENDLY VERSION!

Usage

```
centrality(graph, alpha = 1, posfun = abs, pkg, all.shortest.paths=FALSE)
```

Arguments

graph	A "qgraph" object obtained from qgraph
alpha	The tuning parameter. Defaults to 1.
posfun	A function that converts positive and negative values to only positive. Defaults to the absolute value.
pkg	Package to use. Either "qgraph" or "igraph". Defaults to "qgraph" for directed networks and "igraph" for undirected networks.
all.shortest.paths	Logical if all shortest paths should be returned. Defaults to FALSE. Setting this to true can greatly increase computing time if pkg = "igraph".

Details

This function computes and returns the in and out degrees, closeness and betweenness as well as the shortest path lengths and shortest paths between all pairs of nodes in the graph. For more information on these statistics, see Opsahl, Agneessens and Skvoretz (2010).

These statistics are only defined for positive edge weights, and thus negative edge weights need to be transformed into positive edge weights. By default, this is done by taking the absolute value.

The algorithm used for computing the shortest paths is the well known "Dijkstra's algorithm" (Dijkstra, 1959). The algorithm has been implemented in R, which can make this function take several minutes to run for large graphs (over 100 nodes). A future version of qgraph will include a compiled version to greatly speed up this function.

Value

A list containing:

OutDegree	A vector containing the outward degree of each node.
InDegree	A vector containing the inward degree of each node.
Closeness	A vector containing the closeness of each node.
Betweenness	A vector containing the betweenness of each node
ShortestPathLengths	A matrix containing the shortest path lengths of each pairs of nodes. These path lengths are based on the inverse of the absolute edge weights raised to the power alpha.
ShortestPaths	A matrix of lists containing all shortest path lengths between all pairs of nodes. Use double square brackets to index. E.g., if the list is called 'res', <code>res\$ShortestPaths[[i,j]]</code> gives a list containing all shortest paths between node i and j.

Author(s)

Sacha Epskamp (mail@sachaepskamp.com)

References

Opsahl, T., Agneessens, F., Skvoretz, J. (2010). Node centrality in weighted networks: generalizing degree and shortest paths. *Soc Netw.* 32:245–251.

Dijkstra, E.W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik* 1, 269–271.

See Also

[qgraph](#)

Examples

```
set.seed(1)
adj <- matrix(sample(0:1,10^2,TRUE,prob=c(0.8,0.2)),nrow=10,ncol=10)
Q <- qgraph(adj)

centrality(Q)
```

centrality and clustering plots

Centrality and Clustering plots and tables

Description

These functions can be used to facilitate interpreting centrality and clustering coefficients. The plot functions use ggplot2 (Wickham, 2009). The table functions create a long format table which can easily be plotted in ggplot2.

Usage

```
centralityPlot(..., labels, standardized = TRUE, relative = FALSE, include,
               theme_bw = TRUE, print = TRUE)
clusteringPlot(..., labels, standardized = TRUE, relative = FALSE, include,
               signed = FALSE, theme_bw = TRUE, print = TRUE)
centralityTable(..., labels, standardized = TRUE, relative = FALSE)
clusteringTable(..., labels, standardized = TRUE, relative = FALSE,
                signed = FALSE)
```

Arguments

...	Objects usable in the <code>getWmat</code> generic, such as <code>qgraph</code> objects and weights matrices. Can also be lists containing these objects. Graphs in a list will be plotted in the same panel as different lines and graphs in separate arguments will be plotted in separate panels.
labels	A vector overwriting the labels used
standardized	Logical, should all measures be standardized?
relative	Logical, should all measures be scaled relative to the largest value?
include	A vector of measures to include. if missing all measures available will be included.
signed	Logical indicating if signed clustering coefficients should be plotted.
theme_bw	Adds the ggplot2 black and white theme to the plot
print	If TRUE, the plot is sent to the print command and returned invisible, if FALSE the plot is returned normally. Needed to include plots in e.g., pdf files.

Author(s)

Sacha Epskamp <mail@sachaepskamp.com> \& Jolanda Kossakowski

References

H. Wickham. ggplot2: elegant graphics for data analysis. Springer New York, 2009.

centrality_auto	<i>Automatic centrality statistics of graphs</i>
-----------------	--

Description

This function can be used on several kinds of graphs to compute several node centrality statistics and the edge-betweenness centrality. The input graph can be an adjacency matrix, a weight matrix, an edgelist (weighted or unweighted), a [qgraph](#) object or an [igraph](#) object.

Usage

```
centrality_auto(x)
```

Arguments

x	A graph. Can be a qgraph object, an igraph object, an adjacency matrix, a weight matrix and an edgelist, or a weighted edgelist.
---	--

Details

The function recognizes whether the network is unweighted vs. weighted, undirected vs. directed, and connected vs. disconnected and computes a set of centrality indices that is best suited for that particular kind of network. Edge signs are always disregarded, while edge weights and directions, if present, are considered in the computation of the indices. If the network is disconnected, closeness centrality is computed only considering the largest component (notice that this is different from what function [centrality](#) does).

If x is unweighted and directed, then the indegree, the outdegree, the node betweenness centrality, the closeness centrality, and the edge betweenness centrality are computed. If x is unweighted and undirected, then the degree, the node betweenness centrality, the closeness centrality, and the edge betweenness centralities are computed. If x is weighted and directed, then the instrength and the outstrength (same as indegree and outdegree, but considering weights), the node betweenness centrality, the closeness centrality, and edge betweenness centralities are computed. If x is weighted and undirected, then the strength, the node betweenness centrality, the closeness centrality, and edge betweenness centralities are computed. Additionally, the shortest path length between each pair of nodes is also computed for all the kinds of networks.

Value

A list containing:

`node.centrality`

A dataframe that includes the node centrality statistics. A subset of the following centrality indices is included, depending on the input network: Degree, InDegree, OutDegree, Strength, InStrength, OutStrength, Betweenness, and Closeness.

`ShortestPathLengths`

A matrix containing the shortest path lengths of each pairs of nodes. These path lengths are based on the inverse of the absolute edge weights.

edge.betweenness centrality

The edge betweenness centrality statistic (Newman & Girvan, 2004). Edges are ordered by their decreasing centrality.

Author(s)

Giulio Costantini (giulio.costantini@unimib.it), Sacha Epskamp (mail@sachaepskamp.com)

References

Newman, M. E. J., Girvan, M. (2004). Finding and evaluating community structure in networks. *Physical Review E* 69(026113).

Costantini, G., Epskamp, S., Borsboom, D., Perugini, M., Möttus, R., Waldorp, L., Cramer, A. O. J., State of the aRt personality research: A tutorial on network analysis of personality data in R. Manuscript submitted for publication.

See Also

[qgraph](#), [centrality](#)

Examples

```
set.seed(1)
adj <- matrix(sample(0:1,10^2,TRUE,prob=c(0.8,0.2)),nrow=10,ncol=10)
Q <- qgraph(adj)
centrality_auto(Q)
# notice that a value NA is returned for the closeness centrality of nodes 3 and 9, which are not
# strongly connected to the largest component of the network (3 cannot reach other nodes, 9 cannot
# be reached).
```

clustcoef_auto

Local clustering coefficients.

Description

Compute local clustering coefficients, both signed and unsigned and both for weighted and for unweighted networks.

Usage

```
clustcoef_auto(x, thresholdWS = 0, thresholdON = 0)
clustWS(x, thresholdWS=0)
clustZhang(x)
clustOnnela(x, thresholdON=0)
```

Arguments

<code>x</code>	An undirected graph. Can be a <code>qgraph</code> object, an <code>igraph</code> object, an adjacency matrix, a weight matrix and an edgelist, or a weighted edgelist.
<code>thresholdWS</code>	The threshold used to binarize a weighted network <code>x</code> to compute the binary clustering coefficients <code>clustWS</code> and <code>signed_clustWS</code> . Edges with weights lower than <code>thresholdWS</code> in absolute value are zeroed. For unweighted networks, <code>thresholdWS = 0</code> is the suggested value.
<code>thresholdON</code>	In the computation of Onnela's clustering coefficient <code>clustOnnela</code> , edge of weights lower than <code>thresholdON</code> in absolute value are excluded. The value <code>thresholdON = 0</code> (i.e., no edge is excluded) is generally suggested also for weighted networks.

Details

`clustWS` computes the clustering coefficient for unweighted networks introduced by Watts & Strogatz (1998) and the corresponding signed version (Costantini & Perugini, in press). `ClustZhang` computes the clustering coefficient for weighted networks introduced by Zhang & Horvath (2005) and the corresponding signed version (Costantini & Perugini, in press). `clustOnnela` computes the clustering coefficient for weighted networks introduced by Onnela et al. (2005) and the corresponding signed version (Costantini & Perugini, in press). `clustering_auto` automatically recognizes the kind of the input network `x` (weighted vs. unweighted, signed vs. unsigned) and computes a subset of indices according to the kind of the network: signed indices are not computed for unsigned networks and weighted indices are not computed for unweighted networks. However the unsigned indices are computed for signed networks, by considering the absolute value of the weights, and the unweighted indices are computed for weighted networks, after a binarization according to the parameter `thresholdWS`. `clustering_auto` computes also the weighted clustering coefficient by Barrat et al. (2004), relying on function `transitivity` from package `igraph`. For the computation of the local clustering coefficient, a node must have at least two neighbors: for nodes with less than two neighbors `NaN` is returned.

Value

A dataframe that includes one or more of the following indices.

<code>clustWS</code>	The Watts & Strogatz's (1998) unweighted clustering coefficient
<code>signed_clustWS</code>	The signed version of the Watts & Strogatz's clustering coefficient
<code>clustZhang</code>	The Zhang & Horvath's (2005) weighted clustering coefficient
<code>signed_clustZhang</code>	The signed version of the Zhang & Horvath's (2005) clustering coefficient
<code>clustOnnela</code>	The Onnela et al.'s (2005) clustering coefficient
<code>signed_clustOnnela</code>	The signed version of the Onnela et al.'s (2005) clustering coefficient
<code>clustBarrat</code>	The Barrat et al.'s (2004) weighted clustering coefficient

Warning

The function requires an undirected network. To convert a directed network to undirected use for instance function `upper.tri` (see examples).

Note

Part of the code has been adapted from package WGCNA (Langfelder & Horvath, 2008).

Author(s)

Giulio Costantini (giulio.costantini@unimib.it), Sacha Epskamp (mail@sachaepskamp.com)

References

- Barrat, A., Barthelemy, M., Pastor-Satorras, R., & Vespignani, A. (2004). The architecture of complex weighted networks. In Proc. Natl. Acad. Sci. USA 101 (pp. 3747-3752).
- Costantini, G., Perugini, M. (in press), Generalization of Clustering Coefficients to Signed Correlation Networks
- Langfelder, P., & Horvath, S. (2008). WGCNA: an R package for weighted correlation network analysis. BMC Bioinformatics, 9, 559.
- Onnela, J. P., Saramaki, J., Kertesz, J., & Kaski, K. (2005). Intensity and coherence of motifs in weighted complex networks. Physical Review E, 71(6), 065103.
- Watts, D. J., & Strogatz, S. H. (1998). Collective dynamics of "small-world" networks. Nature, 393(6684), 440-442.
- Zhang, B., & Horvath, S. (2005). A general framework for weighted gene co-expression network analysis. Statistical Applications in Genetics and Molecular Biology, 4(1).

See Also

[centrality_auto](#)

Examples

```
set.seed(1)
# generate a random (directed) network:
net_ig <- igraph::erdos.renyi.game(n=8, p.or.m=.4, type="gnp", directed=TRUE)

# convert it to an adjacency matrix:
net <- as.matrix(igraph::get.adjacency(net_ig, type="both"))

# convert it to a signed and weighted network:
net <- net*matrix(rnorm(ncol(net)^2), ncol=ncol(net))

# make it undirected:
net[upper.tri(net)] <- t(net)[upper.tri(net)]
clustcoef_auto(net)
```

cor_auto

*Automatically compute an appropriate correlation matrix***Description**

This is mainly a wrapper around Lavaan function [lavCor](#) (Rosseel, 2012) to compute a correlation matrix based on psychoric, polyserial and/or Pearson correlations. The wrapper removes all factors and searches for possible ordinal variabes. A variable is classified as ordinal if it is either ordered or if it consist of at most 7 unique integer values. After computing the correlations an additional check will be performed to see if the correlation matrix is positive definite.

Usage

```
cor_auto(data, select, detectOrdinal = TRUE, ordinalLevelMax = 7, npn.SKEPTIC = FALSE,
         forcePD = TRUE, missing = "pairwise", verbose = TRUE)
```

Arguments

data	A data frame
select	Variables to select from the data frame (as used in subset)
detectOrdinal	Logical, should ordinal variables be detected? If FALSE only variables that are ordered are treated as ordinal variables
ordinalLevelMax	Integer specyfyng the amount of unique integer values a variable should have to be classified as ordinal
npn.SKEPTIC	Logical, should the Nonparanormal SKEPTIC from the huge package be applied if the data is continuous? See huge.npn (Zhao, Liu, Roeder, Lafferty and Wasserman, 2014)
forcePD	If TRUE the function checks if the correlation matrix is positive definite. If the matrix is not positive definite nearPD from the Matrix package will be used (Bates and Maechler, 2014).
missing	Corresponds to the missing argument in lavCor
verbose	Logical, should information be printed to the console?

Value

A correlation matrix

Author(s)

Sacha Epskamp <mail@sachaepskamp.com>

References

- Yves Rosseel (2012). lavaan: An R Package for Structural Equation Modeling. Journal of Statistical Software, 48(2), 1-36. URL <http://www.jstatsoft.org/v48/i02/>.
- Tuo Zhao, Han Liu, Kathryn Roeder, John Lafferty and Larry Wasserman (2014). huge: High-dimensional Undirected Graph Estimation. R package version 1.2.6. <http://CRAN.R-project.org/package=huge>
- Douglas Bates and Martin Maechler (2014). Matrix: Sparse and Dense Matrix Classes and Methods. R package version 1.1-3. <http://CRAN.R-project.org/package=Matrix>

Examples

```
## Not run:
### Examples from lavCor (lavaan): ###

library("lavaan")

# Holzinger and Swineford (1939) example
HS9 <- HolzingerSwineford1939[,c("x1", "x2", "x3", "x4", "x5",
                                "x6", "x7", "x8", "x9")]

# Pearson correlations
cor_auto(HS9)

# ordinal version, with three categories
HS9ord <- as.data.frame( lapply(HS9, cut, 3, labels=FALSE) )

# polychoric correlations, two-stage estimation
cor_auto(HS9ord)

## End(Not run)
```

EBICglasso

Compute Gaussian graphical model using graphical lasso based on extended BIC criterium.

Description

This function uses the [glasso](#) package (Friedman, Hastie and Tibshirani, 2011) to compute a sparse gaussian graphical model with the graphical lasso (Friedman, Hastie & Tibshirani, 2008). The tuning parameter is chosen using the Extended Bayesian Information criterium (EBIC) described by Foygel & Drton (2010).

Usage

```
EBICglasso(S, n, gamma = 0.5, penalize.diagonal = FALSE, nlambda = 100,
           lambda.min.ratio = 0.1, returnAllResults = FALSE, checkPD = TRUE,
           penalizeMatrix, ...)
```

Arguments

<code>S</code>	A covariance or correlation matrix
<code>n</code>	Sample size used in computing <code>S</code>
<code>gamma</code>	EBIC tuning parameter. 0.5 is generally a good choice. Setting to zero will cause regular BIC to be used.
<code>penalize.diagonal</code>	Should the diagonal be penalized?
<code>nlambda</code>	Number of lambda values to test.
<code>lambda.min.ratio</code>	Ratio of lowest lambda value compared to maximal lambda
<code>returnAllResults</code>	If TRUE this function does not return a network but the results of the entire glasso path.
<code>checkPD</code>	If TRUE, the function will check if <code>S</code> is positive definite and return an error if not. It is not advised to use a non-positive definite matrix as input as (a) that can not be a covariance matrix and (b) glasso can hang if the input is not positive definite.
<code>penalizeMatrix</code>	Optional logical matrix to indicate which elements are penalized
<code>...</code>	Arguments sent to glasso

Details

The glasso is run for 100 values of the tuning parameter logarithmically spaced between the maximal value of the tuning parameter at which all edges are zero, `lambda_max`, and `lambda_max/100`. For each of these graphs the EBIC is computed and the graph with the best EBIC is selected. The partial correlation matrix is computed using [wi2net](#) and returned.

Value

A partial correlation matrix

Author(s)

Sacha Epskamp <mail@sachaepskamp.com>

References

- Friedman, J., Hastie, T., & Tibshirani, R. (2008). Sparse inverse covariance estimation with the graphical lasso. *Biostatistics*, 9(3), 432-441. Chicago
- Jerome Friedman, Trevor Hastie and Rob Tibshirani (2011). *glasso*: Graphical lasso-estimation of Gaussian graphical models. R package version 1.7. <http://CRAN.R-project.org/package=glasso>
- Foygel, R., & Drton, M. (2010, November). Extended Bayesian Information Criteria for Gaussian Graphical Models. In *NIPS* (pp. 604-612). Chicago
- Revelle, W. (2014) *psych*: Procedures for Personality and Psychological Research, Northwestern University, Evanston, Illinois, USA, <http://CRAN.R-project.org/package=psych> Version = 1.4.4.
- Douglas Bates and Martin Maechler (2014). *Matrix*: Sparse and Dense Matrix Classes and Methods. R package version 1.1-3. <http://CRAN.R-project.org/package=Matrix>

Examples

```
## Not run:
### Using bfi dataset from psych ###
library("psych")
data(bfi)

# Compute correlations:
CorMat <- cor_auto(bfi[,1:25])

# Compute graph with tuning = 0 (BIC):
BICgraph <- EBICglasso(CorMat, nrow(bfi), 0)

# Compute graph with tuning = 0.5 (EBIC)
EBICgraph <- EBICglasso(CorMat, nrow(bfi), 0.5)

# Plot both:
layout(t(1:2))
BICgraph <- qgraph(BICgraph, layout = "spring", title = "BIC", details = TRUE)
EBICgraph <- qgraph(EBICgraph, layout = "spring", title = "EBIC")

# Compare centrality and clustering:
layout(1)
centralityPlot(list(BIC = BICgraph, EBIC = EBICgraph))
clusteringPlot(list(BIC = BICgraph, EBIC = EBICgraph))

## End(Not run)
```

FDRnetwork

Model selection using local False Discovery Rate

Description

This function is a wrapper around [fdrtool](#) to easily compute a correlation or partial correlation network in which all nonsignificant edges are set to zero.

Usage

```
FDRnetwork(net, cutoff = 0.1, method = c('lfdr', 'pval', 'qval'))
```

Arguments

net	A correlation or partial correlation matrix
cutoff	The cutoff value to use. The edges of which the value of the first element of method are higher than the cutoff are removed. Thus, by default, edges with a local false discovery rate of higher than 0.1 are removed from the graph.
method	The method to use with the cutoff. Can be 'lfdr' for the local false discovery rate, 'pval' for the p-value of 'qval' for the q-value.

Details

method = 'lfdr' could result in a very sparse network, so also looking at other values is advisable.

Author(s)

Sacha Epskamp <mail@sachaepskamp.com>

References

Bernd Klaus and Korbinian Strimmer. (2014). fdrtool: Estimation of (Local) False Discovery Rates and Higher Criticism. R package version 1.2.12. <http://CRAN.R-project.org/package=fdrtool>

Examples

```
## Not run:
### Using bfi dataset from psych ###
library("psych")
data(bfi)

### CORRELATIONS ###
# Compute correlations:
CorMat <- cor_auto(bfi[,1:25])

# Run local FDR:
CorMat_FDR <- FDRnetwork(CorMat)

# Number of edges remaining:
mean(CorMat_FDR[upper.tri(CorMat_FDR,diag=FALSE)]!=0)

# None, so might use different criterion:
CorMat_FDR <- FDRnetwork(CorMat, method = "pval")

# Compare:
L <- averageLayout(CorMat, CorMat_FDR)

layout(t(1:2))
qgraph(CorMat, layout = L, title = "Correlation network",
        maximum = 1, cut = 0.1, minimum = 0, esize = 20)
qgraph(CorMat_FDR, layout = L, title = "Local FDR correlation network",
        maximum = 1, cut = 0.1, minimum = 0, esize = 20)

# Centrality:
centralityPlot(list(cor=CorMat, fdr = CorMat_FDR))

### PARTIAL CORRELATIONS ###
# Partial correlation matrix:
library("parcor")
PCorMat <- cor2pcor(CorMat)

# Run local FDR:
```

```

PCorMat_FDR <- FDRnetwork(PCorMat, cutoff = 0.1, method = "pval")

# Number of edges remaining:
mean(PCorMat_FDR[upper.tri(PCorMat_FDR, diag=FALSE)]!=0)

# Compare:
L <- averageLayout(PCorMat, PCorMat_FDR)

layout(t(1:2))
qgraph(PCorMat, layout = L, title = "Partial correlation network",
        maximum = 1, cut = 0.1, minimum = 0, esize = 20)
qgraph(PCorMat_FDR, layout = L, title = "Local FDR partial correlation network",
        maximum = 1, cut = 0.1, minimum = 0, esize = 20)

# Centrality:
centralityPlot(list(cor=PCorMat, fdr = PCorMat_FDR))

## End(Not run)

```

findGraph

Search for optimal correlation or partial correlation network

Description

This function can be used to find an optimal correlation or partial correlation network according to extended BIC (EBIC; Foygel and Drton, 2010). The functions [fitCovGraph](#) and [fitConGraph](#) from the `ggm` package are used in computing these networks (Marchetti, Drton and Sadeghi, 2014).

Usage

```
findGraph(S, n, type = "cor", gamma = 0.5, method = c('stepup', 'stepdown', 'brute'),
          reverseSteps = TRUE, startSig = TRUE)
```

Arguments

<code>S</code>	A sample covariance or correlation matrix. Or a data frame, in which case cor_auto will be used.
<code>n</code>	The sample size
<code>type</code>	"cor" for estimating a correlation network or "pcor" for estimating a partial correlation network
<code>gamma</code>	The EBIC tuning parameter
<code>method</code>	"brute" for brute force search (testing all possible models), "stepup" for stepwise up model search and "stepdown" for stepwise down model search.
<code>reverseSteps</code>	Logical. If method is "stepup" or "stepdown", should the stepping be reversed if a minimum is found? For example, if in stepwise up search a best model is found, should the search be continued by looking at if different edges could be deleted?

startSig Logical. If TRUE the initial model in if method is "stepup" or "stepdown" is the model in which all edges that are insignificant using Holm adjustment are deleted. Otherwise, "stepup" will start with an empty graph and "stepdown" with a fully connected graph.

Details

Due to the length of computing these models, [EBICglasso](#) should be preferred in larger datasets.

Value

A (partial) correlation matrix

Author(s)

Sacha Epskamp <mail@sachaepskamp.com>

References

Foygel, R., & Drton, M. (2010). Extended bayesian information criteria for gaussian graphical models. In *Advances in Neural Information Processing Systems* (pp. 604-612). Chicago

Giovanni M. Marchetti, Mathias Drton and Kayvan Sadeghi (2014). *ggm: A package for Graphical Markov Models*. R package version 2.0. <http://CRAN.R-project.org/package=ggm>

getWmat	<i>Obtain a weights matrix</i>
---------	--------------------------------

Description

This function extracts a weights matrix from various kinds of objects.

Usage

```
## S3 method for class 'matrix'
getWmat(x,nNodes,labels, directed = TRUE,...)
## S3 method for class 'data.frame'
getWmat(x,nNodes,labels, directed = TRUE,...)
## S3 method for class 'igraph'
getWmat(x,labels,...)
## S3 method for class 'qgraph'
getWmat(x,directed,...)
```

Arguments

x	An input object
nNodes	Number of Nodes
labels	A vector specifying the labels of each node
directed	Logical indicating if the graph should be directed
...	Ignored

Value

A weights matrix

Author(s)

Sacha Epskamp <mail@sachaepskamp.com>

makeBW

A qgraph plot can be understood in black and white

Description

Plot a qgraph network that can be understood also in black and white or grayscale. Positive lines are full and negative ones are dashed. Nodes colors are associated to unique motifs. Up to 12 different motifs are supported at the moment.

Usage

```
makeBW(x, colorlist = NA, plot = TRUE)
```

Arguments

x	A qgraph object
colorlist	Optional: a vector of colors. See details.
plot	logical: if FALSE, the network is not plotted.

Details

If no colorlist is specified, each color is randomly associated to one of the motifs. Specifying colorlist serves for (a) assigning colors to a specific motif, because the first color in the vector will always be associated to the first motif (this can be used e.g., for being consistent across plots), or (b) for associating motifs only to some of the colors, but not to others, since only in colors in the colorlist are associated to motifs if a colorlist is specified.

Value

Silently returns a qgraph object "x" in which two new elements are present, "\$graphAttributes\$Nodes\$density" and "\$graphAttributes\$Nodes\$angles", which affect how the nodes are plotted. Can also be further customized and then re-plotted using plot(x).

Author(s)

Giulio Costantini

Examples

```
set.seed(1)
x <- cor(matrix(rnorm(25), nrow = 5))
colors <- c("red", "red", "blue", "blue", "white")

# colored qgraph plot
qg <- qgraph(x, colors = colors)

# randomly assing motifs to colors (notice that white nodes stay white)
makeBW(qg)
# associate a motif only to one of the colors
makeBW(qg, colorlist = c("blue"))
# define an order, which allows to choose motifs
makeBW(qg, colorlist = c("blue", "red"))
makeBW(qg, colorlist = c("red", "blue"))
```

mat2vec

Weights matrix to vector

Description

Converts a weights matrix to a vector of weights. If the matrix is symmetrical only upper triangle values are returned in the vector.

Usage

```
mat2vec(x, diag = FALSE, tol = 1e-10)
```

Arguments

x	A weights matrix
diag	Logical: should diagonal values be included?
tol	Tolerance level

Author(s)

Sacha Epskamp <mail@sachaepskamp.com>

plot.qgraph	<i>Plot method for "qgraph"</i>
-------------	---------------------------------

Description

Plots an object created by [qgraph](#).

Usage

```
## S3 method for class 'qgraph'
plot(x, ...)
```

Arguments

x	A "qgraph" object
...	Not used

Details

If the result of [qgraph](#) is stored, such as `Graph <- qgraph(...)`, the plot can be recreated in two ways. `qgraph(Graph, ...)` reruns `qgraph` with the same arguments used in the original call except those restated in the dots. For example `qgraph(Graph, shape = "square")` will recreate the same plot but now use square nodes instead of circular. `plot(Graph)` will NOT rerun `qgraph` but simply plot the `qgraph` object. This means that now specific graph attributes can be changed before plotting.

More specific, `qgraph(Graph)` will base the new plot on the `Arguments` element of the `qgraph` object and `plot(qgraph)` will base the new plot on the `graphAttributes` element of `qgraph`.

Author(s)

Sacha Epskamp (mail@sachaepskamp.com)

print.qgraph	<i>Print edgelist</i>
--------------	-----------------------

Description

This function prints the edgelist of a "qgraph" object

Usage

```
## S3 method for class 'qgraph'
print(x, ...)
```

Arguments

x	A "qgraph" object
...	These arguments are not used

Author(s)

Sacha Epskamp (mail@sachaepskamp.com)

See Also

[qgraph](#)

qgraph	<i>qgraph</i>
--------	---------------

Description

This is the main function of qgraph which automatically creates an appropriate network and sends it to the plotting method.

Usage

```
qgraph( input, ... )
```

Arguments

input	Can be either a weights matrix or an edgelist. Can also be an object of class "sem" (sem), "mod" (sem), "lavaan" (lavaan), "principal" (psych), "loadings" (stats), "factanal" (stats), "graphNEL" (Rgraphviz), "pcAlgo" (pcalg), "huge" (huge), "select" (huge) or the output of glasso
...	Any additional arguments described below. Also a list with class "qgraph" can be added that contains any of these arguments (this is returned invisibly by the function)

Details

Because of the amount of arguments the usage of the qgraph function has been reduced by using the ... method for clarity. This does mean that arguments need to be specified by using their exact name. For instance, to specify color="red" you can not use col="red".

Important to note is that qgraph needs to compute in many graphs where the border of nodes are in the plotting area. If the graph is manually rescaled (such as through the "zoom" option in RStudio) the plotting area is changed. This means that the computed location of the border of nodes is no longer valid if the nodes are to remain perfectly square or circular. To overcome this, the usePCH argument can be used. If this argument is set to FALSE nodes will be plotted as polygons meaning they will rescale with rescaling the graph (circles can become ovals) and not have perfect resolution in PDF files. If usePCH is set to TRUE a default plotting symbol is used meaning the graph can not

be rescaled but the node will look good in PDF. By default, qgraph sets usePCH to TRUE if it detects the graph is stored in a file.

While the usePCH argument makes graphs rescalable it is not a perfect solution. It is highly recommended to NOT RESCALE PLOTTING AREAS when using qgraph, or to rerun qgraph after the plotting area is rescaled. This means using save graph option from RStudio should be avoided in favor of the filetype argument in qgraph

Value

qgraph returns (invisibly) a 'qgraph' object containing:

Edgelist	A list containing for each edge the node of origin, node of destination, weight and whether the edge is directed and bidirectional.
Arguments	A list containing the arguments used in the qgraph call.
plotOptions	A list containing numerous options used in the plotting method.
graphAttributes	A list containing numerous attributes for nodes, edges and the entire graph
layout	A matrix containing the layout used in the plot
layout.orig	A matrix containing the original (unscaled) layout.

Important additional arguments

layout This argument controls the layout of the graph. "circle" places all nodes in a single circle, "groups" gives a circular layout in which each group is put in separate circles and "spring" gives a force embedded layout. It also can be a matrix with a row for each node and x and y coordinates in the first and second column respectively. Defaults to "circular" in weighted graphs without a groups list, "groups" in weighted graphs with a groups list, and "spring" in unweighted graphs. Can also be a function from the igraph package.

groups An object that indicates which nodes belong together. Can be a list in which each element is a vector of integers identifying the numbers of the nodes that belong together, or a factor.

minimum Edges with absolute weights under this value are not shown (but not omitted). Defaults to 0. Can also be set to "sig" to only show significant edges for graph = "association" and graph = "concentration"). Significance level is controlled by alpha and bonf arguments

maximum qgraph regards the highest of the maximum or highest absolute edge weight as the highest weight to scale the edge widths too. To compare several graphs, set this argument to a higher value than any edge weight in the graphs (typically 1 for correlations).

cut In weighted graphs, this argument can be used to cut the scaling of edges in width and color saturation. Edges with absolute weights over this value will have the strongest color intensity and become wider the stronger they are, and edges with absolute weights under this value will have the smallest width and become vaguer the weaker the weight. If this is set to NULL, no cutoff is used and all edges vary in width and color. Defaults to NULL for graphs with less than 20 nodes. For larger graphs the cut value is automatically chosen to be equal to the maximum of the 75th quantile of absolute edge strengths or the edge strength corresponding to 2n-th edge strength (n being the number of nodes.)

details Logical indicating if minimum, maximum and cutoff score should be printed under the graph. Defaults to FALSE.

threshold A numeric value that defaults to 0. Edges with absolute weight that are not above this value are REMOVED from the network. This differs from minimum which simply changes the scaling of width and color so that edges with absolute weight under minimum are not plotted/invisible. Setting a threshold influences the spring layout and centrality measures obtained with the graph whereas setting a minimum does not. In the case of correlation (graph = "association") or partial correlation (graph = "concentration") networks this argument can also be given a string to omit insignificant edges. See description of this argument in the next section (Additional options for correlation/covariance matrices).

Additional options for correlation/covariance matrices

graph Type of graph to be made when a correlation or covariance matrix is used as input. Setting this to other values than "default" will check if the matrix is a correlation or covariance matrix; if the matrix is not positive definite [nearPD](#) from the Matrix package will be used. Options are:

"association" or "cor" Plots a correlation network. Runs [cov2cor](#) if input is detected to be a covariance matrix and plots the input as is

"concentration" or "pcor" Plots a partial correlation network, using [cor2pcor](#) from the parcor package (Kraemer, Schaefer and Boulesteix, 2009) on the input matrix

"glasso" Will run [EBICglasso](#) to obtain an optimal sparse estimate of the partial correlation matrix using the glasso package (Friedman, Hastie and Tibshirani, 2011)

Outdated and limited supported options are "factorial", which will create a graph based on an exploratory factor analysis, and "sig" will transform all correlations in p-values (using the [fdrtool](#) package; Korbinián Strimmer, 2014) and force mode="sig". "sig2" will do the same but show p-values based on negative statistics in shades of orange

threshold In addition to a numeric value to omit edges this argument can also be assigned a string to omit insignificant edges. Note that this REMOVES edges from the network (which influences centrality measures and the spring layout). Can be "sig" to compute significance without correction for multiple testing, "holm", "hochberg", "hommel", "bonferroni", "BH", "BY", "fdr" or "none" which are used directly in the adjust argument in [corr.p](#) of the psych package (Revelle, 2014). In addition, this argument can be assigned "locfdr" in which edges are set to zero if the local FDR is below FDRcutoff. [fdrtool](#) from the [fdrtool](#) package (Klaus and Strimmer, 2014) is used to compute these measures, which is used inside [FDRnetwork](#).

sampleSize The sample-size. Used when graph = "glasso" or minimum = "sig"

tuning A tuning parameter used in estimation. Currently only used when graph = "glasso" and corresponds to the gamma argument

alpha The significance level (defaults to 0.05) to be used for not showing edges if minimum = "sig".

bonf Logical indicating if a bonferonni correction should be applied if minimum = "sig".

FDRcutoff Cutoff used in which partial correlations should be included if graph = "fdr". Defaults to 0.9

Output arguments

mar A vector of the form c(bottom, left, top, right) which gives the margins. Works similar to the argument in [par\(\)](#). Defaults to c(3,3,3,3)

- filetype** A character containing the file type to save the output in. "R" outputs in a new R window, "pdf" creates a pdf file. "svg" creates a svg file (requires RSVGtipsDevice). "tex" creates LaTeX code for the graph (requires tikzDevice). 'jpg', 'tiff' and 'png' can also be used. If this is given any other string (e.g. filetype="") no device is opened. Defaults to 'R' if the current device is the NULL-device or no new device if there already is an open device. A function such as x11 can also be used
- filename** Name of the file without extension
- width** Width of the plot, in inches
- height** Height of the plot, in inches
- normalize** Logical, should the plot be normalized to the plot size. If TRUE (default) border width, vertex size, edge width and arrow sizes are adjusted to look the same for all sizes of the plot, corresponding to what they would look in a 7 by 7 inches plot if normalize is FALSE.
- DoNotPlot** Runs qgraph but does not plot. Useful for saving the output (i.e. layout) without plotting
- plot** Logical. Should a new plot be made? Defaults to TRUE. Set to FALSE to add the graph to the existing plot.
- rescale** Logical. Defines if the layout should be rescaled to fit the -1 to 1 x and y area. Defaults to TRUE. Can best be used in combination with plot=FALSE.
- standAlone** Logical. If filetype="tex" this argument can be used to choose between making the output a standalone LaTeX file or only the codes to include the graph.

Graphical arguments

- Nodes:** These arguments influence the plotting of nodes in qgraph. Most of them can be assigned a single value or a vector with a value for each node.
- color** A vector with a color for each element in the groups list, or a color for each node. Defaults to the background color ("bg" argument, which defaults to "white") without groups list and rainbow(length(groups)) with a groups list.
- vsiz** A value indicating the size of the nodes (horizontal if shape is "rectangle". Can also be a vector of length 2 (nodes are scaled to degree) or a size for each node. Defaults to $8 \cdot \exp(-nNodes/80) + 1$
- vsiz2** A value indicating the vertical size of the nodes where the shape is "rectangle". Can also be a vector of length 2 (nodes are scaled to degree) or a size for each node. Defaults to the value of 'vsiz'. If 'vsiz' is not assigned this value is used as a scalar to 'vsiz' (e.g., vsiz2 = 1/2 would result in rectangled nodes where the height is half the default width)
- node.width** Scalar on the default value of 'vsiz'. Defaults to 1.
- node.height** Scalar on the default value of 'vsiz2'. Defaults to 1.
- borders** Logical indicating if borders should be plotted, defaults to TRUE.
- border.color** Color vector indicating colors of the borders. Is repeated if length is equal to 1. Defaults to "black"
- border.width** Controls the width of the border. Defaults to 2 and is comparable to 'lwd' argument in 'points'.
- shape** A character containing the shape of the nodes. "circle", "square", "triangle" and "diamond" are supported. In addition, can be a name of an element of polygonList to plot the corresponding polygon (not recommended for large graphs), which by default includes

shapes "ellipse" and "heart" Can also be a vector with a shape for each node. Defaults to "circle".

polygonList A list containing named lists for each element to include polygons to lookup in the shape argument. Each element must be named as they are used in shape and contain a list with elements x and y containing the coordinates of the polygon. By default ellipse and heart are added to this list. These polygons are scaled according to vsize and vsize2

vTrans Transparency of the nodes, must be an integer between 0 and 255, 255 indicating no transparency. Defaults to 255

subplots A list with as elements R expressions or NULL for each node. If it is an R expression it is evaluated to create a plot for the node.

subpars List of graphical parameters to be used in the subplots

subplotbg Background to be used in the subplots. If missing inherits from 'background' argument.

images A character vector of length 1 or the same length as the number of nodes indicating the file location of PNG or JPEG images to use as nodes. Can be NA to not plot an image as node and overwrites 'subplots'

noPar Set to TRUE to not have qgraph run the par function. Useful when sending qgraph plots as subplots using subplots.

pastel Logical, should default colors (for groups or edge equality constraints) be chosen from pastel colors? If TRUE then rainbow_hcl is used.

rainbowStart A number between 0 and 1 indicating the offset used in rainbow functions for default node coloring.

usePCH Logical indicating if nodes should be drawn using polygons or base R plotting symbols. Defaults to TRUE if more than 50 nodes are used in the graph or if the graph is stored in a file. See details.

node.resolution Resolution of the nodes if usePCH=FALSE. Defaults to 100

title String with a title to be drawn in the topleft of the plot.

preExpression A parsable string containing R codes to be evaluated after opening a plot and before drawing the graph.

postExpression A parsable string containing R codes to be evaluated just before closing the device.

diag Should the diagonal also be plotted as edges? defaults to FALSE. Can also be "col" to plot diagonal values as vertex colors.

Node labels: These arguments influence the plotting of node labels in qgraph. Most of them can be assigned a single value or a vector with a value for each node.

labels If FALSE, no labels are plotted. If TRUE, order in weights matrix is used as labels. This can also be a vector with a label for each node. Defaults for graphs with less than 20 nodes to a 3 character abbreviation of the columnnames and rownames if these are identical or else to TRUE. If a label contains an asterisk (e.g. "x1*") then the asterisk will be omitted and the label will be printed in symbol font (use this for Greek letters). Can also be a list with a label as each element, which can be expressions for more advanced mathematical annotation.

label.cex Scalar on the label size.

label.color Character containing the color of the labels, defaults to "black"

label.prop Controls the proportion of the width of the node that the label rescales to. Defaults to 0.9.

label.norm A single string that is used to normalize label size. If the width of the label is lower than the width of the hypothetical label given by this argument the width of label given by this argument is used instead. Defaults to "OOO" so that every label up to three characters has the same fontsize.

label.scale Logical indicating if labels should be scaled to fit the node. Defaults to TRUE.

label.font Integer specifying the label font of nodes. Can be a vector with value for each node

Edges: These arguments influence the plotting of edges qgraph. Most of them can be assigned a single value, a vector with a value per edge when an edgelist is used as input or a matrix containing values for each edge when a weights matrix is used as input.

esize Size of the largest edge (or what it would be if there was an edge with weight maximum). Defaults to $15 * \exp(-nNodes/90) + 1$ for weighted graphs and 2 for unweighted graphs. In directed graphs these values are halved.

edge.width Scalar on 'esize' and 'asize' arguments to make edges wider with a single argument. 'esize' is multiplied with this value and 'asize' with the square root of this value.

edge.color Color of edges. Can be either a single value to make all edges the same color, a matrix with a color for each edge (when using a weights matrix) or a vector with a color for each edge (when using an edgelist). NA indicates that the default color should be used. Note that unless fade=FALSE colors still fade to white corresponding to their strength

posCol Color of positive edges. Can be a vector of two to indicate color of edges under 'cut' value and color of edges over 'cut' value. If 'fade' is set to TRUE the first color will be faded the weaker the edge weight is. If this is only one element this color will also be used for edges stronger than the 'cut' value. Defaults to c("#009900", "darkgreen")

negCol Color of negative edges. Can be a vector of two to indicate color of edges under 'cut' value and color of edges over 'cut' value. If 'fade' is set to TRUE the first color will be faded the weaker the edge weight is. If this is only one element this color will also be used for edges stronger than the 'cut' value. Defaults to c("#BF0000", "red")

unCol Color to indicate the default edge color of unweighted graphs. Defaults to "#808080".

probCol Color of the probability edges. Defaults to "blue". Only used when probabilityEdges = TRUE

probabilityEdges Logical, do edges indicate probabilities? If this is set to TRUE posCol is overwritten by probCol. Mainly implemented for automatic generation of graphs

colFactor Exponent of transformation in color intensity of relative strength. Defaults to 1 for linear behavior.

trans In weighted graphs: logical indicating if the edges should fade to white (FALSE) or become more transparent (TRUE; use this only if you use a background). In directed graphs this is a value between 0 and 1 indicating the level of transparency. (also used as 'transparency')

fade if TRUE (default) and if 'edge.color' is assigned, transparency will be added to edges that are not transparent (or for which no transparency has been assigned) relative to the edge strength, similar if 'trans' is set to TRUE.

loop If diag=TRUE, this can be used to scale the size of the loop. defaults to 1.

lty Line type, see 'par'

edgeConnectPoints This argument specifies the point for each edge to which it connects to a node, in radians. Can be either a matrix with a row for each edge and two columns: The first column indicates the connection point of the source of the edge and the second column specifies the connection point of the destination of the edge. Can also be an array with a row and column for each node two slices which indicate the source and destination of the edge connecting the two nodes.

Edge Curvature: These arguments control the curvature of edges. Most of them can be assigned a single value, a vector with a value per edge when an edgelist is used as input or a matrix containing values for each edge when a weights matrix is used as input.

curve A value indicating how strongly edges should be curved. Either a single value, a vector (edgelist input) with a value for each edge or a matrix (weights matrix input). NA indicates default curve behavior should be used, which only curves edges if there are multiple edges between two nodes.

curveAll Logical, indicating if all edges should be curved with the value of the 'curve' or only edges between nodes that have share multiple edges.

curveDefault The default curvature. Defaults to 1.

curveShape The shape of the curve, as used in `xspline`. Defaults to -1.

curveScale Logical, should curve scale with distance between nodes. Defaults to TRUE. If FALSE, the curve can be exactly determined. Recommended to set to TRUE for graphs and FALSE for diagrams.

curvePivot Quantile to pivot curves on. This can be used to, rather than round edges, make straight edges as curves with "knicks" in them. Can be logical or numeric. FALSE (default) indicates no pivoting in the curved edges, a number indicates the quantile (and one minus this value as quantile) on which to pivot curved edges and TRUE indicates a value of 0.1.

curvePivotShape The shape of the curve around the pivots, as used in `xspline`. Defaults to 0.25.

parallelEdge Logical, set to TRUE to draw parallel straight edges rather than curved edges when there are multiple edges between two nodes. Can be a vector with value per edge for edgelists or a matrix with a value per edge for weights matrices.

parallelAngle The distance in radians an edge is shifted if `parallel=TRUE`. Can be set to NA (default) to determine based on number of edges between two nodes. Can be a vector with value per edge for edgelists or a matrix with a value per edge for weights matrices.

parallelAngleDefault The default value for `parallelAngle`, indicating the angle of the edge furthest from the center. Defaults to $\pi/6$

Edge Labels: These arguments influence the plotting of edge labels qgraph. Most of them can be assigned a single value, a vector with a value per edge when an edgelist is used as input or a matrix containing values for each edge when a weights matrix is used as input.

edge.labels If FALSE, no edge labels are plotted. If TRUE, numerical edge weights are printed on the edges. This can also be a vector with a label for each edge. Defaults to FALSE. If a label contains an asterisk (e.g. "y1*") then the asterisk will be omitted and the label will be printed in symbol font (use this for Greek letters). Can also be a list with a label as each element, which can be expressions for more advanced mathematical annotation.

edge.label.cex Either a single number or a number per edge used as a scalar of the edge label size. Defaults to 1.

edge.label.bg Either a logical or character vector/matrix. Indicates the background behind edge labels. If TRUE (default) a white background is plotted behind each edge label. If FALSE no background is plotted behind edge labels. Can also be a single color character, a vector or matrix of color vectors for each edge.

edge.label.position Vector of numbers between 0 and 1 controlling the relative position of each edge label. Defaults to 0.5 for placing edge labels at the middle of the edge.

edge.label.font Integer specifying the label font of edges. Can be a vector or matrix with value for each node

Layout: Arguments concerning the placement of nodes, in combination with 'layout'.

repulsion Scalar on the default repulse radius in the spring layout. Defaults to 1. Setting this argument to lower values (e.g., 0.5) will cause nodes in the spring layout to repulse each other less. This is especially useful if a few unconnected nodes cause the giant component to visually be clustered too much in the same place.

layout.par A list of arguments passed to `qgraph.layout.fruchtermanreingold` when `layout = "spring"` or to an `igraph` function when such a function is assigned to 'layout'. Defaults to `list(repulse.rad = nNodes^(repulsion), layout = "spring" and list()` otherwise.

layout.control A scalar on the size of the circles created with the circular layout.

aspect Should the original aspect ratio be maintained if `rescale = TRUE`? Defaults to `FALSE`. Set this to `TRUE` to keep the aspect ratio of the original layout (e.g. result from `layout="spring"`).

rotation A vector that can be used to rotate the circles created with the circular layout. Must contain the rotation in radian for each group of nodes. Defaults to zero for each group.

Legend: Arguments to control the legend placed on the right side of the graph.

legend Logical value indicating if a legend should be plotted. Defaults to `TRUE` if a groups object or `nodeNames` is supplied

legend.cex Scalar of the legend. defaults to 1

legend.mode Character, either "groups" indicating the legend should be based on the groups object, or "names" indicating the legend should be based on the `nodeNames` object.

GLratio Relative size of the graph compared to the layout. Defaults to 2.5

layoutScale A vector with a scalar for respectively the x and y coordinates of the layout (which default plotting area is from -1 to 1 on both x and y axis). Setting this to e.g. `c(2,2)` would make the plot twice as big. Use this in combination with 'layoutOffset' and 'plot' arguments to define the graph placement on an existing plot.

layoutOffset A vector with the offset to the x and coordinates of the center of the graph (defaults to (0,0)). Use this in combination with 'layoutScale' and 'plot' arguments to define the graph placement on an existing plot.

nodeNames Names for each node, can be used to plot a legend next to the plot that links the node labels to node names.

Background: These arguments control the background of the plot

bg If this is `TRUE`, a background is plotted in which node colors cast a light of that color on a black background. Can also be a character containing the color of the background Defaults to `FALSE`

bgcontrol The higher this is, the less light each node gives if `bg=TRUE`. Defaults to 6.

bgres square root of the number of pixels used in `bg=TRUE`, defaults to 100.

General graphical arguments:

pty See 'par'

gray Logical, set to `TRUE` to plot the graph in grayscale colors

tooltips A vector with tooltips for each node, only used when `filetype='svg'` or `filetype='tex'`

overlay Logical, should a Venn-diagram like overlay be plotted? If `TRUE` then for each group a `x%` confidence region is plotted for the X and Y position, using `ellipse`

overlaySize Specifies the size of the overlay ellipses. Corresponds to the confidence level (default is 0.5)

font Integer specifying the default font for node and edge labels

Arguments for directed graphs

- directed** Logical indicating if edges are directed or not. Can be TRUE or FALSE to indicate if all edges are directed, a logical vector (when using edgelist) or a logical matrix (when using weights matrix)
- arrows** A logical indicating if arrows should be drawn, or a number indicating how much arrows should be drawn on each edge. If this is TRUE, a simple arrow is plotted, if this is a number, arrows are put in the middle of the edges.
- arrowAngle** Angle of the arrowhead, in radians. Defaults to $\pi/8$ for unweighted graphs and $\pi/4$ for weighted graphs.
- asize** Size of the arrowhead. Defaults to $2 \cdot \exp(-nNodes/20) + 2$.
- open** Logical indicating if open (TRUE) or closed (FALSE) arrowheads should be drawn.
- bidirectional** If this is TRUE, Then directional edges between nodes that have two edges between them are not curved. Defaults to FALSE. Can also be a logical vector (when using edgelist) or a logical matrix (when using weights matrix)

Arguments for graphs based on significance values

- mode** This argument defines the mode used for coloring the edges. The default, "strength" assumes each edge weight indicates the strength of connection centered around and makes positive edges green and negative edges red. If this is set to "sig" then the edge weights are assumed to be significance values and colored accordingly. This can also include negative values, which will be interpreted as p-values based on negative statistics.
- alpha** The significance level (defaults to 0.05) to be used for not showing edges if minimum = "sig", or if Graph = "sig" a vector of max 4 elements indicating the alpha level cutoffs. Defaults to $c(0.0001, 0.001, 0.01, 0.05)$
- sigScale** The function used to scale the edges if mode="sig". Defaults to $\$function(x)0.8 \cdot (1-x)^{(\log(0.4/0.8, 1-0.05))}$
- bonf** Logical indicating if a bonferonni correction should be applied if minimum = "sig" or mode="sig"

Arguments for plotting scores on nodes

- scores** This argument can be used to plot scores of an individual on the test. Should be a vector with the scores for each item. Currently this can only be integer values (e.g. \ LIKERT scales).
- scores.range** Vector of length two indicating the range of the scores, if scores is assigned.

Arguments for manually defining graphs

- mode** The mode argument (see section on significance graph arguments) can also be used to make the weights matrix correspond directly to the width of the edges (as in lwd of plot()). To do this, set mode to "direct".
- edge.color** This argument can be used to overwrite the colors. Can be either a single value to make all edges the same color, a matrix with a color for each edge (when using a weights matrix) or a vector with a color for each edge (when using an edgelist). NA indicates that the default color should be used. Note that unless fade=FALSE colors still fade to white corresponding to their strength

Arguments for knots (tying together edges)

- knots** This argument can be used to tie edges together in their center, which can be useful to, for example, indicate interaction effects. This argument can be assigned a list where each element is a vector containing the edge numbers that should be knotted together. Another option is to assign the argument a integer vector (for edgelist) or a matrix (for weight matrices) with 0 indicating edges that should not be tied together, and increasing numbers indicating each knot.
- knot.size** The size of the knots. Can be of length one or a vector with the size of each knot. Similar to 'vsize'. Defaults to 1.
- knot.color** The color of the knots. Can be of length one or a vector with the size of each knot. Defaults to NA, which will result in a mix of the knotted edge colors.
- knot.borders** Logical indicating if a border should be plotted around the knot. Can be of length one or a vector with the size of each knot. Works similar to 'borders'. Defaults to FALSE
- knot.border.color** Color of the knot borders. Can be of length one or a vector with the size of each knot. Works similar to 'border.color'. Defaults to "black"
- knot.border.width** Width of the knot borders. Can be of length one or a vector with the size of each knot. Works similar to 'border.width'. Defaults to 1

Arguments for bars

- bars** A list with for each node containing either NULL or a vector with values between 0 and 1 indicating where bars should be placed inside the node.
- barSide** Integer for each node indicating at which side the bars should be drawn. 1, 2, 3 or 4 indicating at bottom, left, top or right respectively.
- barColor** A vector with for each node indicating the color of bars. Defaults to the border color of the node.
- barLength** A Vector indicating the relative length of bars of each node compared to the node size. Defaults to 0.5.
- barsAtSide** Logical, should bars be drawn at the side of a node or at its center? Defaults to FALSE.

Additional arguments

- edgelist** Logical, if TRUE 'input' is assumed to be an edgelist, else if FALSE input is assumed to be a weights matrix. By default this is chosen automatically based on the dimensions of 'input' and this argument is only needed if the dimensions are ambiguous (square matrix with 2 or 3 rows/columns)
- weighted** Logical that can be used to force either a weighted graph (TRUE) or an unweighted graph(FALSE).
- nNodes** The number of nodes, only needs to be specified if the first argument is an edge-list and some nodes have no edges
- XKCD** If set to TRUE the graph is plotted in XKCD style based on <http://stackoverflow.com/a/12680841/567015>.

Using qgraph to plot graphs

The first argument of `qgraph()`, `'input'`, is the input. This can be a number of objects but is mainly either a weights matrix or an edgelist. Here we will assume a graph is made of n nodes connected by m edges. `qgraph` is mainly aimed at visualizing (statistical) relationships between variables as weighted edges. In these edge weights a zero indicates no connection and negative values are comparable in strength to positive values. Many (standardized) statistics follow these rules, the most important example being correlations. In the special case where all edge weights are either 0 or 1 the weights matrix is interpreted as an adjacency matrix and an unweighted graph is made.

a weights matrix is a square n by n matrix in which each row and column represents a node. The element at row i and column j indicates the connection from node i to node j . If the weights matrix is symmetrical an undirected graph is made and if the matrix is asymmetrical a directed graph is made.

Alternatively an edgelist can be used. This is a m by 2 matrix (not a list!) in which each row indicates an edge. The first column indicates the number of the start of the edge and the second column indicates the number of the end of the edge. The number of each node is a unique integer between 1 and n . The total number of nodes will be estimated by taking the highest value of the edgelist. If this is incorrect (there are nodes with no edges beyond the ones already specified) the `'nNodes'` argument can be used. If an integer between 1 and n is missing in the edgelist it is assumed to be a node with no edges. To create a weighted graph edge weights can be added as a third column in the edgelist. By default using an edgelist creates a directed graph, but this can be set with the `'directed'` argument.

Interpreting graphs

In weighted graphs green edges indicate positive weights and red edges indicate negative weights. The color saturation and the width of the edges corresponds to the absolute weight and scale relative to the strongest weight in the graph. It is possible to set this strongest edge by using the `'maximum'` argument. When `'maximum'` is set to a value above any absolute weight in the graph that value is considered the strongest edge (this must be done to compare different graphs; a good value for correlations is 1). Edges with an absolute value under the `'minimum'` argument are omitted (useful to keep filesizes from inflating in very large graphs).

In larger graphs the above edge settings can become hard to interpret. With the `'cut'` argument a cutoff value can be set which splits scaling of color and width. This makes the graphs much easier to interpret as you can see important edges and general trends in the same picture. Edges with absolute weights under the cutoff score will have the smallest width and become more colorful as they approach the cutoff score, and edges with absolute weights over the cutoff score will be full red or green and become wider the stronger they are.

Specifying the layout

The placement of the nodes (i.e. the layout) is specified with the `'layout'` argument. It can be manually specified by entering a matrix for this argument. The matrix must have a row for each node and two columns indicating its X and Y coordinate respectively. `qgraph` plots the nodes on a $(-1:1)(-1:1)$ plane, and the given coordinates will be rescaled to fit this plane unless `'rescale'` is FALSE (not recommended). Another option to manually specify the layout is by entering a matrix with more than two columns. This matrix must then consist of zeroes and a number (the order in the weights matrix) for each node indicating its place. For example:

```
0 0 2 0 0
```

```
1 0 3 0 4
```

will place node 2 at the top in the center, node 1 at the bottom left corner, node 3 at the bottom in the center and node 4 at the bottom right corner. It is recommended however that one of the integrated layouts is used. 'layout' can be given a character as argument to accomplish that. layout="circular" will simply place all nodes in a circle if the groups argument is not used and in separate circles per group if the groups argument is used (see next section).

The circular layout is convenient to see how well the data conforms to a model, but to show how the data clusters another layout is more appropriate. By specifying layout="spring" the Fruchterman-reingold algorithm (Fruchterman & Reingold, 1991), which has been ported from the SNA package (Butts, 2010), can be used to create a force-directed layout. In principle, what this function does is that each node (connected and unconnected) repulse each other, and connected nodes also attract each other. Then after a number of iterations (500 by default) in which the maximum displacement of each node becomes smaller a layout is achieved in which the distance between nodes correspond very well to the absolute edge weight between those nodes.

A solution to use this function for weighted graphs has been taken from the igraph package (Csardi G & Nepusz T, 2006) in which the same function was ported from the SNA package. New in qgraph are the option to include constraints on the nodes by fixing a coordinate for nodes or reducing the maximum allowed displacement per node. This can be done with the 'layout.par' argument. For more information see [qgraph.layout.fruchtermanreingold](#).

By default, 'layout' is set to "spring" for unweighted and directed graphs and "circular" otherwise.

Grouping nodes

Grouping nodes (e.g., according to a measurement model) can be specified with the 'groups' argument. This can be a factor or a list in which each element is a vector containing the numbers of nodes that belong together (numbers are taken from the order in the weights matrix). All numbers must be included. If a groups list is specified the "groups" layout can be used to place these nodes together, the nodes in each group will be given a color, and a legend can be plotted (by setting 'legend' to TRUE). The colors will be taken from the 'color' argument, or be generated with the [rainbow](#) function.

Output

By default qgraph will plot the graph in a new R window. However the graphs are optimized to be plotted in a PDF file. To easily create a pdf file set the 'filetype' argument to "pdf". 'filename' can be used to specify the filename and folder to output in. 'height' and 'width' can be used to specify the height and width of the image in inches. By default a new R window is opened if the current device is the NULL-device, otherwise the current device is used (note that when doing this 'width' and 'height' still optimize the image for those widths and heights, even though the output screen size isn't affected, this is especially important for directed graphs!).

Furthermore filetype can also be set to numerous other values. Alternatively any output device in R can be used by simply opening the device before calling qgraph and closing it with dev.off() after calling qgraph.

The graphs can also be outputted in an SVG file using the RSVGTipsDevice package (Plate, 2009). An SVG image can be opened in most browsers (firefox and chrome are recommended), and can be used to display tooltips. Each node can be given a tooltip with the 'tooltips' argument. The function

[qgraph.svg](#) can be used to make a battery of svg pictures with hyperlinks to each other, working like a navigation menu (note, RSVGTipsDevice is a 32-bit only package, so SVG functionality is not available in 64bit versions of R).

Finally, the filetype 'tex' can be used. This uses the tikzDevice package to create a LaTeX file that can then be compiled in your LaTeX compiler to create a pdf file. The main benefit of this over plotting directly in a pdf file is that tooltips can be added which can be viewed in several PDF document readers (Adobe Reader is recommended for the best result).

IMPORTANT NOTE: graphs made in qgraph must be exported programatically using device functions such as pdf() and png(). Manually resizing a graph and using export functions such as the one built into RStudio will give UNSTABLE RESULTS.

Manual specification of color and width

In qgraph the widths and colors of each edge can also be manually controlled. To directly specify the width of each edge set the 'mode' argument to "direct". This will then use the absolute edge weights as the width of each edge (negative values can still be used to make red edges). To manually set the color of each edge, set the 'edge.color' argument to a matrix with colors for each edge (when using a weights matrix) or a vector with a color for each edge (when using an edgelist).

Replotting graphs and reusing layouts

If the result of [qgraph](#) is stored, such as `Graph <- qgraph(...)`, the plot can be recreated in two ways. `qgraph(Graph, ...)` reruns qgraph with the same arguments used in the original call except those restated in the dots. For example `qgraph(Graph, shape = "square")` will recreate the same plot but now use square nodes instead of circular. `plot(Graph)` will NOT rerun qgraph but simply plot the qgraph object. This means that now specific graph attributes can be changed before plotting.

More specific, `qgraph(Graph)` will base the new plot only on the Arguments element of the qgraph object and `plot(qgraph)` will base the new plot on the graphAttributes and plotOptions elements of the qgraph object.

To reuse a layout, use the layout element. e.g., to plot a new graph with the same layout use `qgraph(..., layout = Graph$layout)`

Additional information

By default, edges will be straight between two nodes unless there are two edges between two nodes. To overwrite this behavior the 'bidirectional' argument can be set to TRUE, which will turn two edges between two nodes into one bidirectional edge. 'bidirectional' can also be a vector with TRUE or FALSE for each edge.

To specify the strength of the curve the argument 'curve' can be used (but only in directional graphs). 'curve' must be given a numerical value that represent an offset from the middle of the straight edge through where the curved edge must be drawn. 0 indicates no curve, and any other value indicates a curve of that strength. A value of 0.3 is recommended for nice curves. This can be either one number or a vector with the curve of each edge.

Nodes and edges can be given labels with the 'labels' and the 'edge.labels' arguments. 'labels' can be set to FALSE to omit labels, TRUE (default) to set labels equal to the node number (order in the weights matrix) or it can be a vector with the label for each node. Edge labels can also be set to

FALSE to be omitted (default). If 'edge.labels' is TRUE then the weight of each label is printed. Finally, 'edge.labels' can also be a vector with the label for each edge. If a label (both for edges and nodes) contain an asterisk then the asterisk is omitted and that label is printed in the symbol font (useful to print Greek letters).

A final two things to try: the 'scores' argument can be given a vector with the scores of a person on each variable, which will then be shown using colors of the nodes, And the 'bg' argument can be used to change the background of the graph to another color, or use bg=TRUE for a special background (do set transparency=TRUE when using background colors other then white).

Debugging

If this function crashes for any reason with the filetype argument specified, run:

```
dev.off()
```

To shut down the output device!

Author(s)

Sacha Epskamp <mail@sachaepskamp.com>

References

Carter T. Butts <butts@uci.edu> (2010). sna: Tools for Social Network Analysis. R package version 2.2-0. <http://CRAN.R-project.org/package=sna>

Csardi G, Nepusz T (2006). The igraph software package for complex network research, InterJournal, Complex Systems 1695. <http://igraph.sf.net>

Sacha Epskamp, Angelique O. J. Cramer, Lourens J. Waldorp, Verena D. Schmittmann, Denny Borsboom (2012). qgraph: Network Visualizations of Relationships in Psychometric Data. Journal of Statistical Software, 48(4), 1-18. URL <http://www.jstatsoft.org/v48/i04/>.

Jerome Friedman, Trevor Hastie and Rob Tibshirani (2011). glasso: Graphical lasso-estimation of Gaussian graphical models. R package version 1.7. <http://CRAN.R-project.org/package=glasso>

Bernd Klaus and Korbinian Strimmer. (2014). fdrtool: Estimation of (Local) False Discovery Rates and Higher Criticism. R package version 1.2.12. <http://CRAN.R-project.org/package=fdrtool>

Fruchterman, T. & Reingold, E. (1991). Graph drawing by force-directed placement. Software - Pract. Exp. 21, 1129-1164.

N. Kraemer, J. Schaefer. A.-L. Boulesteix (2009). Regularized Estimation of Large-Scale Gene Regulatory Networks using Gaussian Graphical Models BMC Bioinformatics 10:384

Plate, T. <tplate@acm.org> and based on RSvgDevice by T Jake Luciani <jakeluciani@yahoo.com> (2009). RSVGTipsDevice: An R SVG graphics device with dynamic tips and hyperlinks. R package version 1.0-1.

Revelle, W. (2014) psych: Procedures for Personality and Psychological Research, Northwestern University, Evanston, Illinois, USA, <http://CRAN.R-project.org/package=psych> Version = 1.4.4.

See Also

[cor_auto](#) [qgraph.animate](#) [qgraph.efa](#) [qgraph.pca](#) [qgraph.loadings](#)

Examples

```
## Not run:
### Correlations ###
# Load big5 dataset:
data(big5)
data(big5groups)

# Compute correlation matrix:
big5_cors <- cor_auto(big5, detectOrdinal = FALSE)

# Correlations:
big5Graph <- qgraph(cor(big5),minimum=0.25,groups=big5groups,
  legend=TRUE,borders=FALSE, title = "Big 5 correlations")

# Same graph with spring layout:
qgraph(big5Graph,layout="spring")

# Same graph with different color scheme:
qgraph(big5Graph,posCol="blue",negCol="purple")

### Network analysis ###
### Using bfi dataset from psych ###
library("psych")
data(bfi)

# Compute correlations:
CorMat <- cor_auto(bfi[,1:25])

# Compute graph with tuning = 0 (BIC):
BICgraph <- qgraph(CorMat, graph = "glasso", sampleSize = nrow(bfi),
  tuning = 0, layout = "spring", title = "BIC", details = TRUE)

# Compute graph with tuning = 0.5 (EBIC)
EBICgraph <- qgraph(CorMat, graph = "glasso", sampleSize = nrow(bfi),
  tuning = 0.5, layout = "spring", title = "BIC", details = TRUE)

# Compare centrality and clustering:
centralityPlot(list(BIC = BICgraph, EBIC = EBICgraph))
clusteringPlot(list(BIC = BICgraph, EBIC = EBICgraph))

# Compute centrality and clustering:
centrality_auto(BICgraph)
clustcoef_auto(BICgraph)

### Directed unweighted graphs ###
set.seed(1)
adj=matrix(sample(0:1,10^2,TRUE,prob=c(0.8,0.2)),nrow=10,ncol=10)
qgraph(adj)
title("Unweighted and directed graphs",line=2.5)

# Save plot to nonsquare pdf file:
```

```

qgraph(adj,filetype='pdf',height=5,width=10)

#### EXAMPLES FOR EDGES UNDER DIFFERENT ARGUMENTS ####
# Create edgelist:
dat.3 <- matrix(c(1:15*2-1,1:15*2),,2)
dat.3 <- cbind(dat.3,round(seq(-0.7,0.7,length=15),1))

# Create grid layout:
L.3 <- matrix(1:30,nrow=2)

# Different esize:
qgraph(dat.3,layout=L.3,directed=FALSE,edge.labels=TRUE,esize=14)

# Different esize, strongest edges omitted (note how 0.4 edge is now
# just as wide as 0.7 edge in previous graph):
qgraph(dat.3[-c(1:3,13:15),],layout=L.3,nNodes=30,directed=FALSE,
       edge.labels=TRUE,esize=14)

# Different esize, with maximum:
qgraph(dat.3,layout=L.3,directed=FALSE,edge.labels=TRUE,esize=14,maximum=1)
title("maximum=1",line=2.5)

qgraph(dat.3[-c(1:3,13:15),],layout=L.3,nNodes=30,directed=FALSE,edge.labels=TRUE,
       esize=14,maximum=1)
title("maximum=1",line=2.5)

# Different minimum
qgraph(dat.3,layout=L.3,directed=FALSE,edge.labels=TRUE,esize=14,minimum=0.1)
title("minimum=0.1",line=2.5)

# With cutoff score:
qgraph(dat.3,layout=L.3,directed=FALSE,edge.labels=TRUE,esize=14,cut=0.4)
title("cut=0.4",line=2.5)

# With details:
qgraph(dat.3,layout=L.3,directed=FALSE,edge.labels=TRUE,esize=14,minimum=0.1,
       maximum=1,cut=0.4,details=TRUE)
title("details=TRUE",line=2.5)

# Trivial example of manually specifying edge color and widths:
E <- as.matrix(data.frame(from=rep(1:3,each=3),to=rep(1:3,3),width=1:9))
qgraph(E,mode="direct",edge.color=rainbow(9))

#### Input based on other R objects ####

## Exploratory factor analysis:
big5efa <- factanal(big5,factors=5,rotation="promax",scores="regression")
qgraph(big5efa,groups=big5groups,layout="circle",minimum=0.2,
       cut=0.4,vsize=c(1.5,10),borders=FALSE,vTrans=200,title="Big 5 EFA")

## Principal component analysis:

```

```

library("psych")
big5pca <- principal(cor(big5),5,rotate="promax")
qgraph(big5pca,groups=big5groups,layout="circle",rotation="promax",minimum=0.2,
       cut=0.4,vsize=c(1.5,10),borders=FALSE,vTrans=200,title="Big 5 PCA")

## pcalg
# Example from pcalg vignette:
library("pcalg")
data(gmI)
suffStat <- list(C = cor(gmI$x), n = nrow(gmI$x))
pc.fit <- pc(suffStat, indepTest=gaussCIttest,
            p = ncol(gmI$x), alpha = 0.01)

qgraph(pc.fit)

## glasso:
# Using bfi dataset from psych:
library("psych")
data(bfi)
cor_bfi <- cor_auto(bfi[,1:25])

# Run qgraph:
library("glasso")
bfi_glasso <- glasso(cor_bfi, 0.1)

# Plot:
qgraph(bfi_glasso, layout = "spring")

## Huge (glasso):
library("huge")
bfi_huge <- huge.npn(bfi[,1:25]), method="glasso")

# Auto select optimal and plot:
qgraph(bfi_huge, layout = "spring")

# Manual select and plot:
sel <- huge.select(bfi_huge)
qgraph(sel, layout = "spring")

## End(Not run)

```

qgraph.animate

Animate a growing network

Description

This function is meant to facilitate the creation of animations based on growing networks. Networks are created based on the Fruchterman Reingold algorithm, which is constraint by limiting the

maximum displacement of nodes that are already in the graph.

Usage

```
qgraph.animate(input, ind = NULL, ..., constraint = 10, growth = "order",
  titles = NULL, sleep = 0, smooth = TRUE, plotGraphs = TRUE, progress = TRUE,
  initLayout)
```

Arguments

input	A weights matrix of the graph or a list of weights matrices with different weights of the same graph (see details). See qgraph . Edgelists are currently not supported.
ind	An object that specifies which nodes are included or excluded. See details.
...	Additional arguments sent to qgraph
constraint	The constraint factor of included nodes. See details. Defaults to 10 for a soft-constrained animation. Set to Inf for a hard-constrained animation.
growth	The way nodes are added by default. Set to "order" to include nodes in the order they appear in the weights matrix and to "degree" to include nodes based on their degree (high degree first)
titles	Optional vector with a title for each plot
sleep	Optional value sent to Sys.sleep() for showing the animation in R
smooth	Logical. If set to TRUE smoothing via loess is performed on the layout of all frames.
plotGraphs	Logical. If set to FALSE graphs are not plotted.
progress	Logical. If set to TRUE progress bars are included.
initLayout	An optional n by 2 matrix containing the initial placement of nodes in the animation.

Details

Let n be the number of nodes in total in the graph.

This function is designed to facilitate the production of animations by constraining the Fruchterman Reingold algorithm. Several frames are plotted of (a subset of) the same graph. If a node was already in the graph its maximum displacement per iteration of Fruchterman Reingold is equal to the number of nodes times the inverse of the constraint argument (so by default $n/10$). The higher this constraint value the stricter nodes stay in the same place between plots.

How many and which plots are made are defined by the 'input' and 'ind' arguments. There are two ways to specify the 'input' argument, either by specifying one weights matrix or by specifying a list of weights matrices. In the sections below is explained what both of these methods do and how they are used.

This function, since it can be seen as an expression that makes several plots, works well in combination with the animation package for saving the animation to a wide variety of filetypes.

Value

Invisibly returns a list of all graphs.

Single weights matrix

If 'input' is a single weights matrix then in each frame a subset of the same graph is plotted. This is especially useful for animating the growth of a network. Which nodes are in each frame is determined by the 'ind' argument.

If 'int' is not specified an animation is created in which in each frame a single node is added. This node is either in order of appearance in the weights matrix or by its degree, which is determined with the 'growth' argument.

If 'ind' is a logical vector of length n then the first frame will contain the nodes specified with this vector and all other frames will grow in the same way as explained above (each step one node is added).

If 'ind' is a numeric vector of length n which contains all integers between 1 and n (a single entry per node) then the first frame starts with only the node specified in the first element of the vector and in frame i the ith element is added (each step one node is added).

If 'ind' is a list with numeric vectors as elements containing integers between 1 and n then in frame i the nodes from the ith element of the list will be added. Node numbers that occur multiple times in the list are ignored (they are already added the first time).

Finally, if 'ind' is a logical matrix with n columns and an arbitrary amount of rows, then in frame i only the nodes that are TRUE in row i are included. This is the only way to specify removal of nodes.

List of weights matrices

The 'input' argument can also be given a list of weights matrices if all these matrices have the same dimension (i.e. only the weights differ). If this is done then in frame i the ith weights matrix is used. This is especially useful for animating the change in a graph.

In this case, the 'ind' argument behaves differently. If this argument is not specified then in each frame all nodes are included.

If 'ind' is a logical vector of length n then only one plot is made with the nodes specified with that vector, and only if the length of 'input' is one.

Other methods work in the same way as above. However, if the 'ind' argument indicates a different number of frames than the 'input' argument the function will stop and give an error.

Author(s)

Sacha Epskamp (mail@sachaepskamp.com)

References

Sacha Epskamp, Angelique O. J. Cramer, Lourens J. Waldorp, Verena D. Schmittmann, Denny Borsboom (2012). qgraph: Network Visualizations of Relationships in Psychometric Data. Journal of Statistical Software, 48(4), 1-18. URL <http://www.jstatsoft.org/v48/i04/>.

See Also[qgraph](#)**Examples**

```
## Not run:

## For these examples, first generate a scale free network using preferential attachment:

# Number of nodes:
n <- 100
# Empty vector with Degrees:
Deps <- rep(0, n)
# Empty Edgelist:
E <- matrix(NA, n - 1, 2)
# Add and connect nodes 1 and 2:
E[1, ] <- 1:2
Deps[1:2] <- 1
# For each node, add it with probability proportional to degree:
for (i in 2:(n - 1))
{
  E[i, 2] <- i + 1
  con <- sample(1:i, 1, prob = Deps[1:i]/sum(Deps[1:i]),i)
  Deps[c(con,i+1)] <- Deps[c(con,i+1)] + 1
  E[i, 1] <- con
}

# Because this is an edgelist we need a function to convert this to an adjacency matrix:
E2adj <- function(E,n)
{
  adj <- matrix(0,n,n)
  for (i in 1:nrow(E))
  {
    adj[E[i,1],E[i,2]] <- 1
  }
  adj <- adj + t(adj)
  return(adj)
}

### EXAMPLE 1: Animation of construction algorithm: ###
adjs <- lapply(1:nrow(E),function(i) E2adj(E[1:i,,drop=FALSE],n))
qgraph.animate(adjs,color="black",labels=FALSE,sleep=0.1, smooth = FALSE)
rm(adjs)

### EXAMPLE 2: Add nodes by final degree: ###
adj <- E2adj(E,n)
qgraph.animate(E2adj(E,n),color="black",labels=FALSE,constraint=100,sleep=0.1)

### EXAMPLE 3: Changing edge weights: ###
adjW <- adj*rnorm(n^2)
```

```

adjW <- (adjW + t(adjW))/2
adjs <- list(adjW)
for (i in 2:100)
{
  adjW <- adj*rnorm(n^2)
  adjW <- (adjW + t(adjW))/2
  adjs[[i]] <- adjs[[i-1]] + adjW
}
qgraph.animate(adjs,color="black",labels=FALSE,constraint=100,sleep=0.1)

## End(Not run)

```

qgraph.cfa

CFA using Structural Equation Modelling

Description

This function performs a simple confirmatory factor analysis using `sem` (Fox, 2010) or `lavaan` (Rosseel, 2011).

Usage

```
qgraph.cfa(S, N, groups=NULL, ..., pkg = "sem", labels=NULL, fun = qgraph, opts = list())
```

Arguments

<code>S</code>	A covariance matrix
<code>N</code>	The number of observations
<code>groups</code>	The groups list, see qgraph . This must be a list in which each element is a factor indicating which variables load on the same factor
<code>...</code>	Arguments passed to 'fun'
<code>pkg</code>	A string indicating which package should be used for estimating the model. Currently "lavaan" and "sem" (default) are supported
<code>labels</code>	A vector indicating the label of each variable
<code>fun</code>	A function to which the results are send. Defaults to <code>qgraph</code> , but can be any function that can handle the output (e.g. <code>qgraph.lavaan</code> , <code>qgraph.sem</code> , <code>summary</code> , <code>print</code>). If <code>pkg="sem"</code> then this can also be <code>qgraph.loadings</code> .
<code>opts</code>	A list containing arguments that are sent to either sem or cfa

Details

This function can be used to perform a simple confirmatory factor analysis using regular qgraph input. The function computes a model and then sends it to [sem](#) (sem; Fox, 2010) or [cfa](#) (lavaan; Rosseel, 2011). based on the package used either a "sem" object or a "lavaan" object is returned that can be used for manual inspection or to sent to [qgraph.sem](#) or [qgraph.lavaan](#).

The model that is estimated is a first order factor model in which each variable loads on one factor and the factors are correlated. This model is specified with the 'groups' argument. This must be a list in which each element represents a factor. Each element of the list must be a vector indicating which variables load on the same factor. The model is identified by fixing the first loading of each factor to 1, which should be an identifying restriction if there are at least 4 variables per factor.

The function also sends its results to another function for visualization. If this is [qgraph](#), the default, then a visualization of the standardized coefficients is plotted.

Currently the sem package is better supported in qgraph, but this will change in a future version. Using the lavaan package can greatly reduce computation time.

Value

A "sem" object, see [sem](#)

Author(s)

Sacha Epskamp (mail@sachaepskamp.com)

References

Sacha Epskamp, Angelique O. J. Cramer, Lourens J. Waldorp, Verena D. Schmittmann, Denny Borsboom (2012). qgraph: Network Visualizations of Relationships in Psychometric Data. Journal of Statistical Software, 48(4), 1-18. URL <http://www.jstatsoft.org/v48/i04/>.

John Fox with contributions from Adam Kramer <jfox@mcmaster.ca> and Michael Friendly (2010). sem: Structural Equation Models. R package version 0.9-21. <http://CRAN.R-project.org/package=sem>

See Also

[qgraph](#) [qgraph.sem](#) [qgraph.lavaan](#) [qgraph.loadings](#) [qgraph.semModel](#) [sem](#)

Examples

```
## Not run:
# Simulate dataset:
set.seed(2)
eta<-matrix(rnorm(200*5),ncol=5)
lam<-matrix(rnorm(50*5,0,0.15),50,5)
lam[apply(diag(5)==1,1,rep,each=10)]<-rnorm(50,0.7,0.3)
th<-matrix(rnorm(200*50),ncol=50)
Y<-eta%*%t(lam)+th

# Create groupslist
gr<-list(1:10,11:20,21:30,31:40,41:50)
```

```

# Using "lavaan" package:
res <- qgraph.cfa(cov(Y),N=200,groups=gr,pkg="lavaan",vsize.man=2,vsize.lat=10)

qgraph.lavaan(res,filename="lavaan",legend=FALSE,groups=gr,edge.label.cex=0.6)

# Using "sem" package:
res <- qgraph.cfa(cov(Y),N=200,groups=gr,pkg="sem",vsize.man=2,vsize.lat=10,fun=qgraph.loadings)

qgraph.semModel(res,edge.label.cex=0.6)

qgraph(res,edge.label.cex=0.6)

qgraph.sem(res,filename="sem",legend=FALSE,groups=gr,edge.label.cex=0.6)

### Big 5 dataset ###
data(big5)
data(big5groups)

fit <- qgraph.cfa(cov(big5),nrow(big5),big5groups,pkg="lavaan",opts=list(se="none"),
vsize.man=1,vsize.lat=6,edge.label.cex=0.5)
print(fit)

## End(Not run)

```

qgraph.efa

qgraph.efa

Description

This function performs an Exploratory Factor Analysis (EFA) using the [factanal](#) (stats) function and sends the acquired factor loadings to [qgraph.loadings](#).

Usage

```

qgraph.efa(dat,factors=1,...,rotation="promax",residuals=TRUE,
factorCors=NULL,scores="regression",
corMat=nrow(dat)==ncol(dat) && all(dat==t(dat)))

```

Arguments

<code>dat</code>	A correlation matrix, data matrix or a "factanal" object
<code>factors</code>	The number of factors to extract
<code>rotation</code>	rotation to be used. Can be "varimax", "promax" or "none"
<code>residuals</code>	Logical indicating if residuals should be plotted. Defaults to TRUE
<code>factorCors</code>	Logical indicating if correlations of factors should be extracted and plotted. Defaults to FALSE if a correlation matrix is used and TRUE if a data matrix is used.
<code>...</code>	arguments passed to qgraph.loadings

scores	Method used to extract scores in factanal
corMat	Logical indicating if the 'dat' object is a correlation matrix (TRUE) or data matrix (FALSE)

Author(s)

Sacha Epskamp (mail@sachaepskamp.com)

References

Sacha Epskamp, Angelique O. J. Cramer, Lourens J. Waldorp, Verena D. Schmittmann, Denny Borsboom (2012). qgraph: Network Visualizations of Relationships in Psychometric Data. Journal of Statistical Software, 48(4), 1-18. URL <http://www.jstatsoft.org/v48/i04/>.

See Also

[qgraph](#) [qgraph.pca](#) [qgraph.loadings](#)

Examples

```
## Not run:
data(big5)
data(big5groups)

qgraph.efa(big5,5,groups=big5groups,rotation="promax",minimum=0.2,cut=0.4,
  vsize=c(1,7),borders=FALSE,vTrans=200)

# Tree layout:
qgraph.efa(big5,5,groups=big5groups,rotation="promax",minimum=0.2,cut=0.4,
  vsize=c(1,7),borders=FALSE,layout="tree",width=20,filetype="R")

## End(Not run)
```

qgraph.gui

Graphical user interface for qgraph

Description

THIS FUNCTION HAS TEMPORARILY BEEN REMOVED FROM THE PACKAGE.

Usage

```
qgraph.gui(input, corMat, ...)
```

Arguments

```
input
corMat
...
```

Author(s)

Sacha Epskamp <mail@sachaepskamp.com>

See Also

[qgraph](#)

qgraph.lavaan

qgraph: Structural Equation Modelling

Description

This functions uses a "lavaan" object from the [lavaan](#) package (Rosseel, 2011) and outputs a multi-page pdf file containing path diagram, graphs of the parameter estimates and graphs of the implied and observed covariance and correlation matrices.

Usage

```
qgraph.lavaan(
  fit,
  ...,
  layout="circle",
  groups=NULL,
  vsize.man=3,
  vsize.lat=6,
  filename="qgraph",
  filetype="pdf",
  residuals=TRUE,
  include=1:12,
  curve=0,
  residSize=0.2,
  onefile=TRUE,
  width=12,
  height=8,
  titles=TRUE)
```

Arguments

fit	A " lavaan " object containing the fit of a SEM model (obtained from e.g. sem and cfa)
...	arguments passed to qgraph . This is both for the path diagram and for the correlation/covariance plots.
layout	The layout used for the path diagram. Can be "tree", "spring", "circle" and "springtree"
groups	An optional list containing the measurement model, see qgraph
vsize.man	Size of the manifest variables in the path diagram

vsize.lat	Size of the latent variables in the path diagram
filename	Name of the file (will be extended with the filetype)
filetype	The filetype to be used. Can be "pdf" to make a pdf (default) or anything else to plot in R. More filetypes will be supported in a future version.
residuals	Omitting residuals is currently not supported for qgraph.lavaan, leave this to TRUE
include	A vector indicating which panels should be included in the output
curve	Numerical value indicating the curve of edges that are on the same level in the "tree" layout, See details. This represent an offset from the middle of the straight edge through where the curved edge must be drawn. 0 indicates no curve, and any other value indicates a curve of that strength. Defaults to 0.2
residSize	Size of the residual arrows
onefile	Logical indicating if a multi-page pdf should be produced. If FALSE each plot will be a new pdf. Use this only with panels=1 and filename="(Arbitrary name)%03d".
width	Width of each panel, in inches
height	Height of each panel, in inches
titles	Logical indicating if titles should be printed

Details

This function uses a "[lavaan](#)" object and outputs a multi-page pdf file. The function reads the 'lavaan' object and creates a residual variable for each variable present in the model. Layout options include a tree-layout (layout="tree") in which each variable is placed as a node on one of four vertical levels. At the bottom are the residuals of the manifest variables placed, Above that the manifest variables, above that the latent variables and at the top the residuals of the latent variables. The nodes are evenly spaced horizontally in order of appearance in the model (residuals are placed at the same horizontal position of their corresponding variable). So the order of specifying in the model defines the order of placement in the path diagram. If the 'residuals' argument is FALSE then residuals are omitted and this diagram will only contain two levels.

Alternatively the 'spring' layout can be used (layout="spring"). This will use the Fruchterman-reingold algorithm (Fruchterman & Reingold, 1991), which has been ported from the 'sna' package (Butts, 2010). This is a force-directed algorithm. It is best to use this in combination with residuals=FALSE. Another option is a circular layout (default), which is the same as the tree except that the levels are placed in inner circles rather than horizontal lines.

Names for variables used in the model specification are passed to the path diagram. To keep the model readable it is advised to limit these names to 3 characters.

Note

This is the first installment of [qgraph.sem](#) for the Lavaan package. This function will likely be changed a lot in future installments.

Author(s)

Sacha Epskamp (mail@sachaepskamp.com)

References

Sacha Epskamp, Angelique O. J. Cramer, Lourens J. Waldorp, Verena D. Schmittmann, Denny Borsboom (2012). qgraph: Network Visualizations of Relationships in Psychometric Data. *Journal of Statistical Software*, 48(4), 1-18. URL <http://www.jstatsoft.org/v48/i04/>.

Carter T. Butts <butts@uci.edu> (2010). sna: Tools for Social Network Analysis. R package version 2.2-0. <http://CRAN.R-project.org/package=sna>

Yves Rosseel <Yves.Rosseel@UGent.be> (2011). lavaan: Latent Variable Analysis. R package version 0.4-7. <http://CRAN.R-project.org/package=lavaan>

Fruchterman, T. & Reingold, E. (1991). Graph drawing by force-directed placement. *Software - Pract. Exp.* 21, 1129-1164.

See Also

[qgraph](#) [qgraph.cfa](#) [qgraph.sem](#) [sem](#)

Examples

```
## Not run:
## The industrialization and Political Democracy Example
# Example from lavaan::sem help file:
require("lavaan")
## Bollen (1989), page 332
model <- '
  # latent variable definitions
  ind60 =~ x1 + x2 + x3
  dem60 =~ y1 + y2 + y3 + y4
  dem65 =~ y5 + equal("dem60=~y2")*y6
           + equal("dem60=~y3")*y7
           + equal("dem60=~y4")*y8

  # regressions
  dem60 ~ ind60
  dem65 ~ ind60 + dem60

  # residual correlations
  y1 ~~ y5
  y2 ~~ y4 + y6
  y3 ~~ y7
  y4 ~~ y8
  y6 ~~ y8
  ,

fit <- sem(model, data=PoliticalDemocracy)

# Plot standardized model (numerical):
qgraph.lavaan(fit,layout="tree",vsize.man=5,vsize.lat=10,
filetype="",include=4,curve=-0.4,edge.label.cex=0.6)

# Plot standardized model (graphical):
qgraph.lavaan(fit,layout="tree",vsize.man=5,vsize.lat=10,
```

```

filetype="",include=8,curve=-0.4,edge.label.cex=0.6)

# Create output document:
qgraph.lavaan(fit,layout="spring",vsize.man=5,vsize.lat=10,
filename="lavaan")

## End(Not run)

```

qgraph.layout.fruchtermanreingold

qgraph.layout.fruchtermanreingold

Description

This is a wrapper for the function that returns the x and y coordinates of the graph based on the Fruchterman Reingold algorithm (Fruchterman & Reingold, 1991), which was ported from the SNA package (Butts, 2010). This function is used in [qgraph](#) and is not designed to be used separately. See details for using constraints in this layout.

Usage

```

qgraph.layout.fruchtermanreingold(edgelist, weights=NULL, vcount=NULL,
niter=NULL, max.delta=NULL, area=NULL, cool.exp=NULL, repulse.rad=NULL,
init=NULL, groups=NULL, rotation=NULL, layout.control=0.5, constraints=NULL)

```

Arguments

edgelist	A matrix with on each row the nodes at the start and the node at the end of each edge.
weights	A vector containing the edge weights.
vcount	The number of nodes.
niter	Number of iterations, default is 500.
max.delta	Maximum displacement, default is equal to the number of nodes.
area	The area of the plot, default is the square of the number of nodes.
cool.exp	Cooling exponent, default is 1.5.
repulse.rad	Repulse radius, defaults to the cube of the number of nodes.
init	Matrix with two columns and a row for each node containing the initial X and Y positions.
groups	See qgraph
rotation	See qgraph
layout.control	See qgraph
constraints	A constraints matrix with two columns and a row for each node containing a NA if the node is free or a fixed value for one of the coordinates.

Details

All arguments for this function can be passed from [qgraph](#) to this function by using the 'layout.par' argument, which must be a list containing the arguments. This can be used to constrain the layout in two ways:

Hard constraints

By using the 'constraints' argument the X and Y positions of each node can be fixed to a certain value. The 'constraint' argument must be given a matrix with two columns and a row for each node. An NA means that that coordinate for that node is free, and a value means it is fixed to that value.

Soft constraints

Soft constraining can be done by varying the 'max.delta' argument. This can be a single number, but also a vector containing the maximum displacement per step for each node. The default value is the number of nodes, so by setting this to a lower value for some nodes the node won't move so much. Use this in combination with the 'init' argument to make sure nodes don't move too much from their initial setup. This can be useful when adding a new node to an existing network and if you don't want the network to completely change.

Author(s)

Sacha Epskamp (mail@sachaepskamp.com)

References

Sacha Epskamp, Angelique O. J. Cramer, Lourens J. Waldorp, Verena D. Schmittmann, Denny Borsboom (2012). qgraph: Network Visualizations of Relationships in Psychometric Data. Journal of Statistical Software, 48(4), 1-18. URL <http://www.jstatsoft.org/v48/i04/>.

Carter T. Butts <butts@uci.edu> (2010). sna: Tools for Social Network Analysis. R package version 2.2-0. <http://CRAN.R-project.org/package=sna>

Fruchterman, T. & Reingold, E. (1991). Graph drawing by force-directed placement. Software - Pract. Exp. 21, 1129-1164.

See Also

[qgraph](#)

Examples

```
## Not run:
# This example makes a multipage PDF that contains images
# Of a building network using soft constraints.

# Each step one node is added with one edge. The max.delta
# decreases the longer nodes are present in the network.

pdf("Soft Constraints.pdf",width=10,height=5)
```

```

adj=adj0=matrix(0,nrow=3,ncol=3)
adj[upper.tri(adj)]=1
Q=qgraph(adj,vsize=3,height=5,width=10,layout="spring",
  esize=1,filetype='',directed=T)
cons=Q$layout
for (i in 1:20)
{
  x=nrow(adj)
  adjN=matrix(0,nrow=x+1,ncol=x+1)
  adjN[1:x,1:x]=adj
  consN=matrix(NA,nrow=x+1,ncol=2)
  consN[1:x,]=cons[1:x,]
  layout.par=list(init=rbind(cons,c(0,0)),
    max.delta=10/(x+1):1,area=10^2,repulse.rad=10^3)
  y=sample(c(x,sample(1:(x),1)),1)
  adjN[y,x+1]=1
  Q=qgraph(adjN,Q,layout="spring",layout.par=layout.par)
  cons=Q$layout
  adj=adjN
}
dev.off()

## End(Not run)

```

qgraph.loadings

qgraph.loadings

Description

This function is a wrapper function for [qgraph](#) designed to visualize factor loadings.

Usage

```
qgraph.loadings( fact, ...)
```

Arguments

<code>fact</code>	A matrix containing factor loadings (items per row, factors per column) or an "loadings" object
<code>...</code>	Additional optional arguments passed to qgraph and special arguments used in this function (described below).

Additional optional arguments

layout If "default" a standard layout for factor models will be made. If this is "circle" the default layout is circled (factors in the centre, items at the edge). No other layouts are currently supported.

vsize A vector where the first value indicates the size of manifest variables and the second value indicates the size of latent variables.

model "reflective" to have arrows go to manifest variables, "formative" to have arrows go to latent variables or "none" (default) for no arrows

crossloadings Logical, if TRUE then for each manifest variable the strongest loading is omitted (default to FALSE).

groups An optional list containing the measurement model, see [qgraph](#)

Fname When there is only one factor, this is its name. If there are more factors, the names in the groups list are used only if the factors can be identified.

resid Values for the residuals

residSize Size of the residuals, defaults to 0.1

factorCors Correlation matrix of the factors

Author(s)

Sacha Epskamp (mail@sachaepskamp.com)

References

Sacha Epskamp, Angelique O. J. Cramer, Lourens J. Waldorp, Verena D. Schmittmann, Denny Borsboom (2012). qgraph: Network Visualizations of Relationships in Psychometric Data. Journal of Statistical Software, 48(4), 1-18. URL <http://www.jstatsoft.org/v48/i04/>.

See Also

[qgraph](#) [qgraph.pca](#) [qgraph.efa](#)

Examples

```
## Not run:
# Load big5 dataset:
data(big5)
data(big5groups)

big5efa <- factanal(big5,factors=5,rotation="promax",scores="regression")
big5loadings <- loadings(big5efa)
qgraph.loadings(big5loadings,groups=big5groups,rotation="promax",minimum=0.2,
cut=0.4,vsize=c(1.5,15),borders=FALSE,vTrans=200)

# Tree layout:
qgraph.loadings(big5loadings,groups=big5groups,rotation="promax",minimum=0.2,
cut=0.4,vsize=c(1.5,15),borders=FALSE,vTrans=200,
layout="tree",width=20,filetype="R")

## End(Not run)
```

qgraph.panel	<i>qgraph.panel</i>
--------------	---------------------

Description

Creates a 4-panel graph. See details. The usage is the same as [qgraph](#)

Usage

```
qgraph.panel(input, ...)
```

Arguments

input	The weights matrix or edgelist. See qgraph . Correlation matrix is recommended.
...	Optional additional arguments (only 'layout' and 'graph' are omitted). See qgraph .

Details

This function will create a 4-panel plot containing four plots useful in analyzing correlation matrices:

1. Association graph with circular layout
2. Association graph with spring layout
3. Concentration graph with spring layout
4. Factorial graph with spring layout

Author(s)

Sacha Epskamp (mail@sachaepskamp.com)

References

Sacha Epskamp, Angelique O. J. Cramer, Lourens J. Waldorp, Verena D. Schmittmann, Denny Borsboom (2012). qgraph: Network Visualizations of Relationships in Psychometric Data. Journal of Statistical Software, 48(4), 1-18. URL <http://www.jstatsoft.org/v48/i04/>.

See Also

[qgraph](#) [qgraph.svg](#)

Examples

```
## Not run:
data(big5)
data(big5groups)

qgraph.panel(cor(big5),groups=big5groups,minimum=0.2,borders=FALSE,vsize=1,cut=0.3)

## End(Not run)
```

qgraph.pca

qgraph.pca

Description

This function performs an Principal Component Analysis (PCA) using the 'princomp' function of the psych package (Revelle, 2010) and sends the acquired factor loadings to [qgraph.loadings](#).

Usage

```
qgraph.pca( cor, factors=1, ..., rotation="promax", factorCors = TRUE)
```

Arguments

cor	A correlation matrix or a "principal" object
factors	The number of factors to extract
...	arguments passed to qgraph.loadings
rotation	rotation to be used. Can be "varimax", "promax" or "none"
factorCors	Logical, should the correlations between factors be plotted? Defaults to TRUE

Author(s)

Sacha Epskamp (mail@sachaepskamp.com)

References

Sacha Epskamp, Angelique O. J. Cramer, Lourens J. Waldorp, Verena D. Schmittmann, Denny Borsboom (2012). qgraph: Network Visualizations of Relationships in Psychometric Data. Journal of Statistical Software, 48(4), 1-18. URL <http://www.jstatsoft.org/v48/i04/>.

Revelle, W. (2010) psych: Procedures for Personality and Psychological Research Northwestern University, Evanston, <http://personality-project.org/r/psych.manual.pdf>, 1.0-93

See Also

[qgraph](#) [qgraph.efa](#) [qgraph.loadings](#)

Examples

```
## Not run:
data(big5)
data(big5groups)

qgraph.pca(cor(big5),5,groups=big5groups,rotation="promax",minimum=0.2,
cut=0.4,vsize=c(1,7),borders=FALSE,vTrans=200)

# Tree layout:
qgraph.pca(cor(big5), 5,groups=big5groups, rotation="promax", minimum=0.2,
  cut=0.4, vsize=c(1.5,7), borders=FALSE, layout="tree", width=20, filetype="R")

## End(Not run)
```

qgraph.sem

qgraph: Structural Equation Modelling

Description

This functions uses a "sem" object from the [sem](#) function (from the sem package; Fox, 2010) and outputs a multi-page pdf file containing goodness of fit indices, path diagram, graphs of the parameter estimates and graphs of the implied and observed covariance and correlation matrices.

Usage

```
qgraph.sem(res, layout="circle", ..., vsize.man=3, vsize.lat=6, filename="qgraph",
filetype="pdf", residuals=TRUE, panels=2, include=1:12, latres=TRUE,
curve=0, residSize=0.2, onefile=TRUE, width = 7, height = 7)
```

Arguments

res	The output from the 'sem' function. See details for extra details on specifying the model
layout	The layout used for the path diagram. Can be "tree", "spring", "circle" and "springtree"
...	arguments passed to qgraph . This is both for the path diagram and for the correlation/covariance plots.
vsize.man	Size of the manifest variables in the path diagram
vsize.lat	Size of the latent variables in the path diagram
filename	Name of the file (will be extended with the filetype)
filetype	The filetype to be used. Can be "pdf" to make a pdf (default) or anything else to plot in R. More filetypes will be supported in a future version.
residuals	Logical indicating if the residuals should be included in the path diagram. If this is FALSE then residual variances will be shown as colors on the nodes. Default is TRUE

panels	The number of panels to be used in the pdf file. Can be 1, 2, 4 or 8
include	A vector indicating which panels should be included in the output
latres	This is currently not supported, leave to TRUE
curve	Numerical value indicating the curve of edges that are on the same level in the "tree" layout, See details. This represent an offset from the middle of the straight edge through where the curved edge must be drawn. 0 indicates no curve, and any other value indicates a curve of that strength. Defaults to 0.2
residSize	Size of the residual arrows
onefile	Logical indicating if a multi-page pdf should be produced. If FALSE each plot will be a new pdf. Use this only with panels=1 and filename="(Arbitrary name)%03d".
width	Width of each panel, in inches
height	Height of each panel, in inches

Details

This function uses a "sem" object and outputs a multi-page pdf file. The function reads the 'sem' file and creates a residual variable for each variable present in the model. The default layout is a tree-layout (layout="tree") in which each variable is placed as a node on one of four vertical levels. At the bottom are the residuals of the manifest variables placed, Above that the manifest variables, above that the latent variables and at the top the residuals of the latent variables. The nodes are evenly spaced horizontally in order of appearance in the model (residuals are placed at the same horizontal position of their corresponding variable). So the order of specifying in the model defines the order of placement in the path diagram. If the 'residuals' argument is FALSE then residuals are omitted and this diagram will only contain two levels.

Alternatively the 'spring' layout can be used (layout="spring"). This will use the Fruchterman-reingold algorithm (Fruchterman & Reingold, 1991) is used, which has been ported from the 'sna' package (Butts, 2010). This is a force-directed algorithm. It is best to use this in combination with residuals=FALSE.

Names for variables and edges used in the model specification are passed to the path diagram. To keep the model readable it is advised to limit these names to 3 characters. If a variable or edge name contains an asterisk, the asterisk is omitted and the name will be printed in the symbol font. This way Greek letters can be used (e.g. the edge name "l*" will make a lambda character). Because the symbol font conveniently uses Arabic numerals, parameter names like beta1 can be created with "*b1" in the model.

Author(s)

Sacha Epskamp (mail@sachaepskamp.com)

References

Sacha Epskamp, Angelique O. J. Cramer, Lourens J. Waldorp, Verena D. Schmittmann, Denny Borsboom (2012). qgraph: Network Visualizations of Relationships in Psychometric Data. Journal of Statistical Software, 48(4), 1-18. URL <http://www.jstatsoft.org/v48/i04/>.

Carter T. Butts <buttsc@uci.edu> (2010). sna: Tools for Social Network Analysis. R package version 2.2-0. <http://CRAN.R-project.org/package=sna>

John Fox with contributions from Adam Kramer <jfox@mcmaster.ca> and Michael Friendly (2010). sem: Structural Equation Models. R package version 0.9-21. <http://CRAN.R-project.org/package=sem>

Fruchterman, T. & Reingold, E. (1991). Graph drawing by force-directed placement. Software - Pract. Exp. 21, 1129-1164.

See Also

[qgraph](#) [qgraph.cfa](#) [qgraph.semModel](#) [sem](#)

Examples

```
## Not run:

require('sem')

# This example is taken from the examples of the sem function.
# Only names were changed to better suit the path diagram.

# ----- Thurstone data -----
# Second-order confirmatory factor analysis, from the SAS manual for PROC CALIS

R.thur <- readMoments(diag=FALSE, names=c('Sen','Voc',
      'SC','FL','4LW','Suf',
      'LS','Ped','LG'))

.828
.776 .779
.439 .493 .46
.432 .464 .425 .674
.447 .489 .443 .59 .541
.447 .432 .401 .381 .402 .288
.541 .537 .534 .35 .367 .32 .555
.38 .358 .359 .424 .446 .325 .598 .452

model.thur <- specifyModel()
F1 -> Sen,          *111, NA
F1 -> Voc,          *121, NA
F1 -> SC,           *131, NA
F2 -> FL,           *141, NA
F2 -> 4LW,          *152, NA
F2 -> Suf,          *162, NA
F3 -> LS,           *173, NA
F3 -> Ped,          *183, NA
F3 -> LG,           *193, NA
F4 -> F1,           *g1,  NA
F4 -> F2,           *g2,  NA
F4 -> F3,           *g3,  NA
Sen <-> Sen,        q*1,  NA
Voc<-> Voc,        q*2,  NA
SC <-> SC,         q*3,  NA
FL <-> FL,         q*4,  NA
```

```
4LW <-> 4LW,          q*5,  NA
Suf<-> Suf,            q*6,  NA
LS <-> LS,              q*7,  NA
Ped<-> Ped,            q*8,  NA
LG <-> LG,              q*9,  NA
F1 <-> F1,              NA,    1
F2 <-> F2,              NA,    1
F3 <-> F3,              NA,    1
F4 <-> F4,              NA,    1

sem.thur <- sem(model.thur, R.thur, 213)

# Run qgraph:
qgraph.sem(sem.thur,filename="Thurstone tree",layout="tree",edge.label.cex=0.6,
  curve=0.4,width=8,height=5)

# Spring layout:
qgraph.sem(sem.thur,filename="Thurstone spring",layout="spring",residuals=FALSE,
  width=5,height=5)
## End(Not run)
```

qgraph.semModel	<i>qgraph: SEM model pathdiagram</i>
-----------------	--------------------------------------

Description

This is a watered down version of [qgraph.sem](#) that only plots a diagram. It is called if a "mod" object is supplied to [qgraph](#)

Usage

```
qgraph.semModel(mod, manifest = NULL, layout = "spring", vsize.man = 3,
  vsize.lat = 6, residuals = TRUE, latres = TRUE, curve = 0.2, residSize = 0.2,
  ...)
```

Arguments

mod	A "mod" object (model of the sem package (Fox; 2010) or a "sem" object
manifest	Vector containing the names of the manifest variables
layout	The layout used for the path diagram. Can be "tree", "spring", "circle" and "springtree". Defaults to "spring"
vsize.man	Size of the manifest variables in the path diagram
vsize.lat	Size of the latent variables in the path diagram
residuals	Logical indicating if the residuals should be included in the path diagram. If this is FALSE then residual variances will be shown as colors on the nodes. Default is TRUE

latres	This is currently not supported, leave to TRUE
curve	Numerical value indicating the curve of edges that are on the same level in the "tree" layout. See details. This represent an offset from the middle of the straight edge through where the curved edge must be drawn. 0 indicates no curve, and any other value indicates a curve of that strength. Defaults to 0.2
residSize	Size of the residual arrows
...	Arguments passed to qgraph

Author(s)

Sacha Epskamp (mail@sachaepskamp.com)

References

Sacha Epskamp, Angelique O. J. Cramer, Lourens J. Waldorp, Verena D. Schmittmann, Denny Borsboom (2012). qgraph: Network Visualizations of Relationships in Psychometric Data. Journal of Statistical Software, 48(4), 1-18. URL <http://www.jstatsoft.org/v48/i04/>.

John Fox with contributions from Adam Kramer <jfox@mcmaster.ca> and Michael Friendly (2010). sem: Structural Equation Models. R package version 0.9-21. <http://CRAN.R-project.org/package=sem>

See Also

[qgraph](#)

Examples

```
## Not run:

require('sem')

# This example is taken from the examples of the sem function.
# Only names were changed to better suit the path diagram.

# ----- Thurstone data -----
# Second-order confirmatory factor analysis, from the SAS manual for PROC CALIS

R.thur <- readMoments(diag=FALSE, names=c('Sen', 'Voc',
    'SC', 'FL', '4LW', 'Suf',
    'LS', 'Ped', 'LG'))
.828
.776 .779
.439 .493 .46
.432 .464 .425 .674
.447 .489 .443 .59 .541
.447 .432 .401 .381 .402 .288
.541 .537 .534 .35 .367 .32 .555
.38 .358 .359 .424 .446 .325 .598 .452

model.thur <- specifyModel()
F1 -> Sen,          *111, NA
```

```
F1 -> Voc,          *121, NA
F1 -> SC,            *131, NA
F2 -> FL,            *141, NA
F2 -> 4LW,           *152, NA
F2 -> Suf,           *162, NA
F3 -> LS,            *173, NA
F3 -> Ped,           *183, NA
F3 -> LG,            *193, NA
F4 -> F1,            *g1,  NA
F4 -> F2,            *g2,  NA
F4 -> F3,            *g3,  NA
Sen <-> Sen,         q*1,   NA
Voc<-> Voc,          q*2,   NA
SC <-> SC,           q*3,   NA
FL <-> FL,           q*4,   NA
4LW <-> 4LW,         q*5,   NA
Suf<-> Suf,          q*6,   NA
LS <-> LS,           q*7,   NA
Ped<-> Ped,          q*8,   NA
LG <-> LG,           q*9,   NA
F1 <-> F1,           NA,     1
F2 <-> F2,           NA,     1
F3 <-> F3,           NA,     1
F4 <-> F4,           NA,     1

# Run qgraph:
qgraph(model.thur)

# Tree layout:
qgraph(model.thur,layout="tree",manifest=c('Sen','Voc','SC','FL','4LW','Suf','LS','Ped','LG'))

## End(Not run)
```

qgraph.svg	qgraph.svg
------------	------------

Description

IMPORTANT NOTE: RSVGTipsDevice is a 32-bit only package, so SVG functionality is not available in 64bit versions of R.

This function makes a series of SVG images with hyperlinks to each other using the RSVGTipsDevice package (Plate, 2009). The arguments are the same as [qgraph](#) except that some arguments can be assigned a vector with multiple options.

Usage

```
qgraph.svg( input, layout=c( "circular", "spring" ),
graph=c( "association", "concentration", "factorial" ),
cut=c( 0, 0.2, 0.3, 0.5), filename="qgraph", title="qgraph output",
nfact=round(ncol(input)/2,0), tooltips=NULL,... )
```

Arguments

input	A weights matrix (correlation matrix is recommended). See qgraph
layout	The layout of the graph, see qgraph , unlike qgraph it is not possible to assign a specific layout.
graph	The graph to be made based on the correlation matrix. See qgraph
cut	The cutoff score. See qgraph
filename	The name of the SVG files.
title	The title that will be printed at the top of the pictures
nfact	The number of factors that will be used in the EFA that makes the factorial graph. See qgraph
tooltips	A vector with for each node the tooltip to be used.
...	Additional arguments that are passed to qgraph

Details

This function works the same as [qgraph](#) except that the arguments 'layout', 'graph' and 'cut' can be given a vector with multiple options. The RSVGTipsDevice package (Plate, 2009) is used to accomplish this. A SVG picture will be created for each combination of the arguments 'layout', 'graph' and 'cut', with hyperlinks between them

Author(s)

Sacha Epskamp (mail@sachaepskamp.com)

References

Sacha Epskamp, Angelique O. J. Cramer, Lourens J. Waldorp, Verena D. Schmittmann, Denny Borsboom (2012). qgraph: Network Visualizations of Relationships in Psychometric Data. Journal of Statistical Software, 48(4), 1-18. URL <http://www.jstatsoft.org/v48/i04/>.

Plate, T. <tplate@acm.org> and based on RSvgDevice by T Jake Luciani <jakeluciani@yahoo.com> (2009). RSVGTipsDevice: An R SVG graphics device with dynamic tips and hyperlinks. R package version 1.0-1.

See Also

[qgraph](#)

Examples

```
## Not run:
#### VISUALIZE CORRELATION MATRIX ####
eta=matrix(rnorm(200*5),ncol=5)
lam=matrix(0,nrow=100,ncol=5)
for (i in 1:5) lam[(20*i-19):(20*i),i]=rnorm(20,0.7,0.3)
eps=matrix(rnorm(200*100),ncol=100)
Y=eta%*%t(lam)+eps

tooltips=paste("item",1:100)
groups=list(1:20,21:40,41:60,61:80,81:100)
names(groups)=paste("Factor",LETTERS[1:5])
# Run qgraph:
qgraph.svg(cor(Y),groups=groups,tooltips=tooltips,vsize=3)
## End(Not run)
```

qgraphAnnotate

Annotates a qgraph object with mouseover tooltips in a HTML file

Description

This function creates a HTML file based on the result from a [qgraph](#) call. The HTML file is made using [sendplot](#)

Usage

```
qgraphAnnotate(graph, ..., fromqgraph = c("labels", "nodeNames", "tooltips", "groups"),
               filename = "qgraph", image.size = "600x600", window.size = image.size,
               legend = FALSE)
```

Arguments

graph	Result from qgraph
...	Named vectors with elements to send to the tooltip. The length of each vector should equal the amount of nodes.
fromqgraph	Vector specifying which arguments from qgraph should be used in the tooltips. Only supported for "labels", "nodeNames", "tooltips" and "groups"
filename	String indicating the root name of the file
image.size	String indicating the size of the file in pixels.
window.size	String indicating the size of the image window in pixels.
legend	Logical, should a legend be plotted? Defaults to FALSE

Details

This function uses the sendplot package (Gaile, Shepherd, Sucheston, Bruno and Manly, 2013).

Value

A string indicating the location of the created HTML file, which can be used with `browseURL`.

Author(s)

Sacha Epskamp <mail@sachaepskamp.com>

References

Daniel P Gaile, Lori A. Shepherd, Lara Sucheston, Andrew Bruno and Kenneth F. Manly (2013). sendplot: Tool for sending interactive plots with tool-tip content.. R package version 4.0.0. <http://CRAN.R-project.org/package=sendplot>

qgraphMixed

Plots a mixed graph with both directed and undirected edges.

Description

This function can be used to plot a network in which each node is connected by at most 3 edges; one undirected edge and two directed edges.

Usage

```
qgraphMixed(undirected, directed, parallel = TRUE, parallelAngle = pi/6,
             diagUndirected = FALSE, diagDirected = TRUE, ltyUndirected = 1, ltyDirected = 1,
             curve = 1, ...)
```

Arguments

<code>undirected</code>	The undirected network weights matrix.
<code>directed</code>	The directed network weights matrix.
<code>parallel</code>	Logical indicating if edges should be plotted parallel or curved.
<code>parallelAngle</code>	See qgraph
<code>diagUndirected</code>	Logical indicating if the diagonal of the undirected edges should be included.
<code>diagDirected</code>	Logical indicating if the diagonal of the directed edges should be included.
<code>ltyUndirected</code>	lty of undirected edges
<code>ltyDirected</code>	lty of directed edges
<code>curve</code>	Curvature of directed edges
<code>...</code>	Arguments sent to qgraph

Author(s)

Sacha Epskamp <mail@sachaepskamp.com>

smallworldness	<i>Compute the small-worldness index.</i>
----------------	---

Description

Compute the small-worldness index (Humphries & Gurney, 2008) relying on the global transitivity of the network (Newman, 2003) and on its average shortest path length.

Usage

```
smallworldness(x, B = 1000, up = 0.995, lo = 0.005)
```

Arguments

x	A graph. Can be a qgraph object, an igraph object, an adjacency matrix, a weight matrix and an edgelist, or a weighted edgelist.
B	The number of random networks.
up	The upper quantile.
lo	the lower quantile.

Details

The function computes the transitivity of the target network and the average shortest path length. Then it computes the average of the same indices on B random networks. The small-worldness index is then computed as the transitivity (normalized by the random transitivity) over the average shortest path length (normalized by the random average shortest path length). The lo and up quantiles of the distribution of the random networks are also returned for both the transitivity and the average shortest path length.

A network can be said "smallworld" if its smallworldness is higher than one (a stricter rule is `smallworldness >= 3`; Humphries & Gurney, 2008). To consider a network as "smallworld", it is also suggested to inspect that the network has a transitivity substantially higher than comparable random networks and that its average shortest path length is similar or higher (but not many times higher) than that computed on random networks. Edge weights, signs and directions are ignored in the computation of the indices.

Value

smallworldness	the "small-worldness" index proposed by Humphries & Gurney (2008)
trans_target	the global transitivity of the target network (Newman, 2003)
averagelength_target	the average shortest path length in the target network
trans_rnd_M	the average transitivity in the B random networks
trans_rnd_lo	the lo quantile of the transitivity in the B random networks
trans_rnd_up	the up quantile of the transitivity in the B random networks

```

averagelength_rnd_M
    the average shortest path length in the B random networks
averagelength_rnd_lo
    the lo quantile of the shortest path length in the B random networks
averagelength_rnd_up
    the up quantile of the shortest path length in the B random networks

```

Note

If a directed network is given as input, an edge between every two nodes *i* and *j* is considered present if there is an arrow either from *i* to *j* or from *j* to *i* or both.

Author(s)

Giulio Costantini (giulio.costantini@unimib.it), Sacha Epskamp (mail@sachaepskamp.com)

References

Costantini, G., Epskamp, S., Borsboom, D., Perugini, M., Mottus, R., Waldorp, L., Cramer, A. O. J., State of the aRt personality research: A tutorial on network analysis of personality data in R. Manuscript submitted for publication.

Humphries, M. D., & Gurney, K. (2008). Network "small-world-ness": a quantitative method for determining canonical network equivalence. *PLoS One*, 3(4), e0002051.

Newman, M. E. J. (2003). The structure and function of complex networks*. *SIAM Review*, 45(3), 167–256.

Examples

```

set.seed(1)
# a regular lattice. Even if the small-worldness is higher than three, the average path length is
# much higher than that of random networks.
regnet<-igraph::watts.strogatz.game(dim=1, size=1000, nei=10, p=0, loops=FALSE, multiple=FALSE)
smallworldness(regnet, B=10)

## Not run:
# a small-world network: the transitivity is much higher than random, the average path length is
# close to that of random networks
swnet<-igraph::watts.strogatz.game(dim=1, size=1000, nei=10, p=.1, loops=FALSE, multiple=FALSE)
smallworldness(swnet, B=10)

# a pseudorandom network: both the average path length and the transitivity are similar to random
# networks.
rndnet<-igraph::watts.strogatz.game(dim=1, size=1000, nei=10, p=1, loops=FALSE, multiple=FALSE)
smallworldness(rndnet, B=10)

## End(Not run)

```

summary.qgraph	<i>Summary method for "qgraph"</i>
----------------	------------------------------------

Description

This function creates a brief summary based on a "qgraph" object.

Usage

```
## S3 method for class 'qgraph'
summary(object, ...)
```

Arguments

object	A "qgraph" object
...	These arguments are not used

Author(s)

Sacha Epskamp (mail@sachaepskamp.com)

See Also

[qgraph](#)

VARglm	<i>Computes a vector autoregressive lag-1 model using GLM</i>
--------	---

Description

This function computes a VAR model using glm.

Usage

```
VARglm(x, family, vars, adjacency, icfun = BIC, ...)
```

Arguments

x	A data frame
family	The family to be used. Defaults to gaussian if data is continuous or binomial if data is binary
vars	Vector of variables to predict. If missing all variables are predicted.
adjacency	Adjacency matrix. If missing full network is estimated
icfun	Information criterium function to be included in the output
...	Arguments used in the icfun

Value

A list containing:

graph	The estimated graph
IC	The information criterium

Author(s)

Sacha Epskamp <mail@sachaepskamp.com>

wi2net

Converts precision matrix to partial correlation matrix

Description

A small function that converts a precision matrix (inverse of covariance matrix) to a partial correlation matrix. This can be done by standardizing the precision matrix and changing the sign of the offdiagonal entries. Many methods exist for obtaining a precision matrix (Such as the glasso package; Friedman, Hastie and Tibshirani, 2011) but the partial correlation matrix is easier interpretable and better usable in qgraph.

Usage

```
wi2net(x)
```

Arguments

x	A precision matrix
---	--------------------

Value

A partial correlation matrix

Author(s)

Sacha Epskamp <mail@sachaepskamp.com>

References

Jerome Friedman, Trevor Hastie and Rob Tibshirani (2011). glasso: Graphical lasso-estimation of Gaussian graphical models. R package version 1.7. <http://CRAN.R-project.org/package=glasso>

Index

*Topic **Correlations**

qgraph, [22](#)

*Topic **Graphs**

qgraph, [22](#)

*Topic **SEM**

qgraph.lavaan, [46](#)

qgraph.sem, [55](#)

*Topic **SVG**

qgraph.svg, [60](#)

*Topic **\textasciitildekw1**

qgraph.animate, [38](#)

*Topic **\textasciitildekw2**

qgraph.animate, [38](#)

*Topic **black**

makeBW, [19](#)

*Topic **centrality**

centrality_auto, [8](#)

*Topic **cfa**

qgraph.cfa, [42](#)

qgraph.lavaan, [46](#)

*Topic **clustering**

clustcoef_auto, [9](#)

*Topic **graphs**

centrality, [5](#)

centrality_auto, [8](#)

*Topic **graph**

qgraph.lavaan, [46](#)

qgraph.sem, [55](#)

*Topic **lavaan**

qgraph.lavaan, [46](#)

*Topic **path diagram**

qgraph.lavaan, [46](#)

qgraph.sem, [55](#)

*Topic **qgraph**

qgraph, [22](#)

qgraph.cfa, [42](#)

qgraph.lavaan, [46](#)

qgraph.sem, [55](#)

qgraph.svg, [60](#)

*Topic **sem**

qgraph.cfa, [42](#)

*Topic **signed**

clustcoef_auto, [9](#)

*Topic **smallworld**

smallworldness, [64](#)

*Topic **transitivity**

smallworldness, [64](#)

*Topic **weighted**

clustcoef_auto, [9](#)

*Topic **white**

makeBW, [19](#)

as.igraph.qgraph, [2](#)

averageLayout, [3](#)

big5, [4](#)

big5groups, [4](#)

centrality, [5](#), [8](#), [9](#)

centrality and clustering plots, [7](#)

centrality_auto, [5](#), [8](#), [11](#)

centralityPlot(centrality and
clustering plots), [7](#)

centralityTable(centrality and
clustering plots), [7](#)

cfa, [42](#), [43](#), [46](#)

clustcoef_auto, [9](#)

clusteringPlot(centrality and
clustering plots), [7](#)

clusteringTable(centrality and
clustering plots), [7](#)

clustOnnela(clustcoef_auto), [9](#)

clustWS(clustcoef_auto), [9](#)

clustZhang(clustcoef_auto), [9](#)

cor_auto, [12](#), [17](#), [35](#)

corr.p, [24](#)

cov2cor, [24](#)

EBICglasso, [13](#), [18](#), [24](#)

ellipse, 29

factanal, 44, 45

FDRnetwork, 15, 24

fdrtool, 15, 24

findGraph, 17

fitConGraph, 17

fitCovGraph, 17

getWmat, 7, 18

glasso, 13, 14

huge.npn, 12

igraph, 8, 10

lavaan, 46, 47

lavCor, 12

loess, 39

makeBW, 19

mat2vec, 20

nearPD, 12, 24

plot.qgraph, 21

print.qgraph, 21

qgraph, 2, 3, 5, 6, 8, 9, 21, 22, 22, 34, 39, 41–43, 45, 46, 48–55, 57–63, 66

qgraph.animate, 35, 38

qgraph.cfa, 42, 48, 57

qgraph.efa, 35, 44, 52, 54

qgraph.gui, 45

qgraph.lavaan, 43, 46

qgraph.layout.fruchtermanreingold, 29, 33, 49

qgraph.loadings, 35, 43–45, 51, 54

qgraph.panel, 53

qgraph.pca, 35, 45, 52, 54

qgraph.sem, 43, 47, 48, 55, 58

qgraph.semModel, 43, 57, 58

qgraph.svg, 34, 53, 60

qgraphAnnotate, 62

qgraphMixed, 63

rainbow, 33

rainbow_hcl, 26

sem, 42, 43, 46, 48, 55, 57

sendplot, 62

smallworldness, 64

subset, 12

summary.qgraph, 66

transitivity, 10

upper.tri, 10

VARglm, 66

wi2net, 14, 67